

게임 인공지능

Game Artificial Intelligence

부제

NPC의 움직임과 판단, 학습을 만드는 13가지 알고리즘 패턴

키워드

게임 AI · FSM · 경로 탐색 · Behavior Tree · GOAP · 강화학습 · Unity

판권 (Copyright Page)

제목: 게임 인공지능

부제: NPC 의 움직임과 판단, 학습을 만드는 13 가지 알고리즘 패턴

저자: 이수미

소속: 배재대학교 소프트웨어학부 게임공학전공

출판사: [출판사명]

출판일: 2026 년 3 월

판차: 제 1 판 1쇄

ISBN: [ISBN 번호]

저작권: Copyright © 2026 by 이수미. All rights reserved.

이 책의 전부 또는 일부를 무단으로 복제, 배포, 전송하는 것은 저작권법에 따라 금지됩니다.

GitHub 리포지토리: <https://github.com/sumilee-pcu/gameAI>

교재 문의: sumilee@pcu.ac.kr

Unity 버전: Unity 2022.3 LTS 이상 권장

개발 환경: Windows 10/11, macOS Monterey 이상

머리말

왜 게임 AI 인가?

게임 속 NPC(Non-Player Character)는 단순한 그래픽 오브젝트가 아닙니다. <팩맨>의 네 마리 유령은 각각 다른 추적 전략으로 플레이어를 압박하고, <더 라스트 오브 어스>의 적 AI 는 엄폐와 협동 전술로 긴장감을 조성하며, <도타 2>의 OpenAI Five 는 세계 최고 수준의 프로 게이머와 대등하게 경쟁합니다. 이 모든 것이 게임 인공지능(Game AI)의 결과입니다.

게임 AI 는 학술적 AI 와는 다른 독특한 목표를 가집니다. 체스 AI 가 "승리"를 추구한다면, 게임 AI 는 "재미"를 추구합니다. 완벽한 정확도보다는 인간적인 실수를, 수학적 최적성보다는 그럴듯한 행동을, 일관성보다는 예측 불가능한 놀라움을 제공합니다. 그렇기 때문에 게임 AI 설계는 알고리즘 구현을 넘어 경험 디자인(Experience Design)의 영역입니다.

이 책의 철학

이 책은 다음 세 가지 원칙으로 구성되었습니다:

1. 이론과 실습의 균형

수학적 정의와 의사코드로 원리를 이해하고, Unity C# 코드로 직접 구현하여 체득합니다. 각 알고리즘은 "왜 필요한가?"에서 시작하여 "어떻게 동작하는가?"를 거쳐 "어떻게 만드는가?"로 이어집니다.

2. 13 가지 알고리즘 패턴

게임 AI 의 방대한 영역을 13 개의 핵심 알고리즘 패턴으로 체계화했습니다. FSM, Boids, A*, Behavior Tree, GOAP, Q-Learning, Perlin Noise 등 업계 표준 기술을 실전 관점에서 다룹니다.

3. 검증 가능한 학습

모든 알고리즘은 Unity 프로젝트로 구현되며, GitHub 리포지토리에서 완성된 코드를 확인할 수 있습니다. 단순히 "이런 게 있다"가 아니라 "내가 만들 수 있다"를 목표로 합니다.

누가 읽어야 하는가?

이 책은 다음과 같은 독자를 대상으로 합니다:

- 게임 프로그래밍 전공 학생: 게임 AI 수업의 주교재 또는 부교재로 활용
- 인디 게임 개발자: 상용 게임 수준의 AI 를 구현하고 싶은 1 인 또는 소규모 팀
- Unity 개발자: AI 기능을 게임에 적용하고 싶으나 어디서부터 시작할지 모르는 개발자
- AI/ML 연구자: 학술적 AI 를 게임 환경에 적용하려는 연구자

선수지식:

- C# 프로그래밍 기초 (클래스, 상속, 인터페이스)
- Unity 기본 사용법 (씬, 게임오브젝트, 컴포넌트)
- 고등학교 수준 수학 (벡터, 확률, 그래프 이론 기초)

학습 후 도달점

이 책을 완료하면 여러분은 다음을 수행할 수 있습니다:

1. 설계: 게임 시나리오에 맞는 AI 시스템을 체계적으로 설계할 수 있습니다.
2. 구현: 13 가지 알고리즘 패턴을 Unity C#로 구현할 수 있습니다.
3. 검증: 성능 측정, 디버깅, 파라미터 튜닝을 통해 AI 를 최적화할 수 있습니다.
4. 통합: 여러 알고리즘을 결합하여 복잡한 게임 AI 시스템을 완성할 수 있습니다.

감사의 말

이 책은 배재대학교 게임공학과에서 2023 년부터 진행해온 게임 관련 강의 노트와 기타 인공지능, 게임엔진 분야 대학 특강 등의 피드백을 기반으로 집필되었습니다. 알고리즘을 직접 구현하며 고민했던 수백 명의 학생들, 그리고 코드 리뷰와 프로젝트로 함께 성장한 모든 분들께 감사드립니다.

또한 게임 AI 분야의 선구자들인 Craig Reynolds(Boids), Ian Millington(AI for Games), Mat Buckland(Programming Game AI by Example), Steve Rabin(Game AI Pro 시리즈) 등의 연구와 저술에 깊은 영향을 받았음을 밝힙니다.

게임 AI 는 끊임없이 진화합니다. 이 책이 여러분의 첫 발걸음이 되기를, 그리고 더 창의적이고 재미있는 게임을 만드는 데 기여하기를 바랍니다.

2026 년 3 월

배재대학교 소프트웨어학부 게임공학전공

겸임교수 이수미

-목차-

1 장 게임 AI 개론

2 장 경로탐색(A*, NavMesh)

3 장 센서와 지각

4 장 FSM

5 장 Behavior Tree

6 장 고급 의사결정(Utility/GOAP)

7 장 이동과 군집

8 장 머신러닝기초

9 장 강화학습 + ML-Agents

10 장 적응형 AI(DDA)

11 장 PCG

12 장 LLM 기반 NPC

13 장 생성 AI 통합

Part I. AI 기초와 에이전트

게임 AI 설계

NPC의 움직임과 판단, 학습을 만드는 13 가지 알고리즘 패턴

Part I 개요

게임 속 NPC(Non-Player Character)는 단순한 그래픽 오브젝트가 아닙니다. 플레이어와 상호작용하고, 상황을 판단하며, 때로는 예측 불가능한 행동으로 놀라움을 선사하는 지능형 에이전트(Intelligent Agent)입니다. Part I에서는 게임 인공지능(Game AI)의 기본 개념을 정립하고, 학술적 AI와 게임 AI의 결정적인 차이점을 이해합니다.

또한 에이전트가 세상을 어떻게 인식하고 행동하는지를 체계적으로 분석하는 PEAS 모델을 학습하며, 수십 년간 업계 표준으로 사용되어 온 유한 상태 기계(Finite State Machine, FSM)의 설계 및 구현 방법을 수학적 기초부터 실전 응용까지 다룹니다.

학습 목표

본 Part 를 학습한 후 여러분은 다음을 수행할 수 있어야 합니다:

- 1. 게임 AI 와 학술적 AI 의 차이를 이해하고, 게임 AI 설계의 핵심 원칙을 설명할 수 있다.
- 2. 지능형 에이전트의 개념을 이해하고 PEAS 모델로 게임 AI 를 체계적으로 설계할 수 있다.
- 3. 오토마타 이론을 기반으로 FSM 의 수학적 정의를 이해하고 상태 전이 다이어그램을 작성할 수 있다.
- 4. FSM 을 다양한 방법(Switch-Case, State Pattern, Table-Driven)으로 구현할 수 있다.
- 5. 계층적 FSM(HFSM)의 필요성을 이해하고 복잡한 AI 를 모듈화하여 설계할 수 있다.
- 6. 실제 게임 사례를 분석하여 FSM 의 장단점을 평가할 수 있다.

Part I 구성

챕터	제목	주요 내용	페이지
Chapter 1	게임 인공지능 개론과 에이전트	AI 정의와 역사, 게임 AI 특수성, PEAS 모델, 에이전트 유형	14p
Chapter 2	유한 상태 기계 (FSM)	오토마타 이론, FSM 설계와 구현, HFSM, 게임 사례 분석	14p

Chapter 1. 게임 인공지능 개론과 에이전트

게임 AI 는 플레이어에게 도전과 재미를 제공하는 핵심 요소입니다. 본 챕터에서는 인공지능의 역사와 정의를 살펴보고, 게임 AI 가 학술적 AI 와 어떻게 다른지, 그리고 지능형 에이전트를 체계적으로 설계하는 방법을 학습합니다.

1.1 인공지능과 게임 AI

1.1.1 AI 의 정의와 역사

정의 1.1 (인공지능)

인공지능(Artificial Intelligence, AI)은 인간의 지능이 필요한 작업—학습, 추론, 문제 해결, 지각, 언어 이해—을 수행할 수 있는 컴퓨터 시스템을 의미합니다.

AI 의 역사적 발전:

시대	주요 이정표	게임 AI 발전
1950 년대	튜링 테스트 제안 (1950)	-
1960-70 년대	전문가 시스템, 논리 추론	<스페이스 인베이더> (1978) - 패턴 기반 이동
1980 년대	기호 AI 의 전성기	<팩맨> (1980) - FSM 기반 고스트 AI
1990 년대	신경망 부활, 강화학습	<하프라이프> (1998) - 협동 전술 AI
2000 년대	머신러닝 보편화	F.E.A.R (2005) - GOAP 기반 동적 계획
2010 년대	딥러닝 혁명	<도타 2> (2016) - OpenAI Five
2020 년대	생성형 AI, LLM	NPC 대화 생성, 프로시저럴 콘텐츠

1.1.2 튜링 테스트와 지능의 척도

정의 1.2 (튜링 테스트)

앨런 튜링이 1950 년 제안한 기계의 지능을 평가하는 실험입니다. 인간 평가자가 기계와 대화할 때, 그것이 기계인지 인간인지 구분할 수 없다면 그 기계는 "지능적"이라고 간주합니다.

게임 AI 에서는 튜링 테스트의 변형인 "믿을 만함(Believability)" 테스트가 중요합니다. 플레이어가 NPC 를 "살아있는 존재"처럼 느끼는가? 이는 완벽한 지능보다 인간다움(Humanness)에 가깝습니다.

예제 1.1: 믿을 만한 AI vs 완벽한 AI

시나리오	완벽한 AI	믿을 만한 AI
FPS 저격수	100% 헤드샷, 즉시 반응	70% 명중률, 0.3 초 반응 지연, 가끔 빗나감
레이싱 상대	완벽한 코너링, 절대 실수 안 함	가끔 코너에서 미끄러짐, 긴장감 있는 접전
체스 AI	항상 최선수, 절대 패배 안 함	플레이어 수준에 맞춰 조절, 역전 기회 제공

평가: 완벽한 AI 는 플레이어를 좌절시키지만, 믿을 만한 AI 는 재미있는 경험을 제공합니다.

1.1.3 게임 AI의 특수성: 재미 vs 최적성

게임 AI는 학술적 AI와 근본적으로 다른 목표를 가집니다.

표 1.1: 학술적 AI vs 게임 AI 비교

비교 항목	학술적 AI (Academic AI)	게임 AI (Game AI)
주요 목표	문제 해결, 최적화, 승리	재미(Fun), 몰입감(Immersion), 적절한 도전
성능 지표	정확도, 승률, 효율성	플레이어 만족도, 재미 지수
연산 자원	대규모 연산 가능 (슈퍼컴퓨터, 클라우드)	제한적 (그래픽, 물리, 사운드와 CPU/GPU 공유)
정보 접근	완전 정보 선호 (체스, 바둑)	불완전 정보 처리 필수 (전장의 안개, 잠입)
실패 허용	실패는 결함	실패는 설계의 일부 (인간적 실수)
예측 가능성	일관성 중시	변화와 놀라움 필요
결과 품질	수학적 최적성	"그럴듯함(Plausibility)" 중시

사례 연구: AlphaGo vs <스타크래프트> AI

- AlphaGo: 이세돌 9단을 4:1로 격파. 목표는 "세계 최강의 바둑 기사를 이기는 것". 수백 개의 GPU를 사용한 대규모 신경망과 몬테카를로 트리 탐색(MCTS). 실수 최소화.
- <스타크래프트> 게임 AI: 플레이어와 비슷한 수준의 APM(분당 행동 수) 제한. 정찰 실패, 유닛 관리 실수를 의도적으로 설계. 플레이어가 전략으로 이길 수 있도록 약점 노출. 목표는 "도전적이지만 이길 수 있는 상대".

핵심 원리:

좋은 게임 AI = 적절한 도전 + 공정성 + 예측 가능성의 균형

너무 쉬우면 지루하고(Boredom), 너무 어려우면 좌절(Frustration)합니다. 게임 AI 는 플레이어를 몰입 상태(Flow State)에 유지해야 합니다.

1.1.4 게임 AI vs 일반 AI

일반 AI 의 분야:

- 기계 학습 (Machine Learning): 데이터로부터 패턴 학습
- 컴퓨터 비전 (Computer Vision): 이미지 인식
- 자연어 처리 (NLP): 언어 이해 및 생성
- 로보틱스 (Robotics): 물리적 행동 제어

게임 AI 의 고유 영역:

- 캐릭터 행동 (Character Behavior): FSM, BT, GOAP
- 경로 탐색 (Pathfinding): A*, NavMesh
- 전술 AI (Tactical AI): 엄폐, 측면 공격, 팀 협동
- 전략 AI (Strategic AI): 자원 관리, 장기 계획
- 애니메이션 AI: 모션 매칭, IK(역운동학)

최근 융합 트렌드:

- 강화학습(RL)을 게임 AI 에 적용: <도타 2> OpenAI Five
- 생성형 AI 로 NPC 대화 생성: ChatGPT 기반 동적 퀘스트
- 머신러닝으로 플레이어 행동 예측: 개인화된 난이도 조절

1.2 지능형 에이전트 이론

게임 속 캐릭터나 NPC 를 체계적으로 설계하기 위해 지능형 에이전트(Intelligent Agent) 모델을 사용합니다.

1.2.1 에이전트의 정의

정의 1.3 (에이전트)

에이전트는 센서(Sensors)를 통해 환경(Environment)을 인식하고, 구동기(Actuators)를 통해 환경에 영향을 미치는 자율적 개체입니다.

에이전트의 기본 루프:

```
while 게임 실행 중:
    percept ← SENSE(environment)    # 센서로 환경 인식
    action ← DECIDE(percept, state)  # 의사결정
    EXECUTE(action)                  # 구동기로 행동 실행
    state ← UPDATE(state, percept)   # 내부 상태 갱신
```

예제 1.2: FPS 게임의 적 AI 에이전트

센서 (Sensors):

- 시야각(FOV) 내 플레이어 감지
- 총소리 청각 감지
- 체력 상태 확인
- 동료 AI 의 통신 메시지

의사결정 (Decision):

- if 플레이어 발견 → 전투 모드
- if 체력 < 30% → 엄폐물 찾기
- if 총소리 들림 → 경계 상태

구동기 (Actuators):

- 이동 (걷기, 뛰기, 웅크리기)
- 사격 (조준, 발사, 재장전)
- 아이템 사용 (수류탄, 치료킷)
- 애니메이션 재생

1.2.2 PEAS 모델

정의 1.4 (PEAS 모델)

PEAS 는 에이전트를 체계적으로 설계하기 위한 프레임워크입니다:

- Performance (성과 지표): 에이전트의 성공을 측정하는 기준
- Environment (환경): 에이전트가 활동하는 공간의 특성
- Actuators (구동기): 환경에 영향을 미치는 행동 수단
- Sensors (센서): 환경 정보를 수집하는 도구

예제 1.3: 잠입 액션 게임 경비병 AI 의 PEAS 모델

요소	내용
Performance	• 침입자 발견 시간 최소화 • 생존 시간 최대화 • 순찰 경로 커버리지
Environment	• 실내 건물 (복도, 방, 계단) • 조명 시스템 (밝음/어두움) • 소음 발생 가능 (발소리, 문 소리) • 다른 경비병 존재 (협동 가능)
Actuators	• 이동 (순찰 경로 따라가기) • 회전 (주변 둘러보기) • 무기 사용 (총, 경보 버튼) • 무전기 통신 (동료에게 알림)
Sensors	• 시각 센서 (FOV 120°, 거리 15m) • 청각 센서 (소음 감지, 거리 20m) • 통신 수신 (동료의 경보 메시지) • 내부 상태 (체력, 탄약)

PEAS 모델의 장점:

- 설계 단계에서 빠진 요소를 점검할 수 있음
- 팀원 간 의사소통 명확화

- AI 범위와 한계를 명시적으로 정의

1.2.3 에이전트 유형

에이전트는 내부 복잡도에 따라 다음과 같이 분류됩니다.

1. 단순 반사 에이전트 (Simple Reflex Agent)

현재의 지각(Percept)에만 반응하여 행동합니다. 메모리나 과거 이력이 없습니다.

```
function SIMPLE_REFLEX_AGENT(percept):  
    state ← INTERPRET_INPUT(percept)  
    rule ← RULE_MATCH(state, rules)  
    action ← rule.ACTION  
    return action
```

장점: 빠르고 단순

단점: 상황 변화에 유연하지 못함, 반복적 패턴

예시:

- <스페이스 인베יד러>의 외계인: 좌우 이동 + 플레이어 방향으로 발사
- 벽돌 깨기의 공: 벽/패들에 부딪히면 반사

2. 모델 기반 반사 에이전트 (Model-based Reflex Agent)

내부 상태(State)를 유지하여 과거 정보를 기억합니다.

```
function MODEL_BASED_REFLEX_AGENT(percept):  
    state ← UPDATE_STATE(state, action, percept, model)  
    rule ← RULE_MATCH(state, rules)
```

```
action ← rule.ACTION  
return action
```

내부 모델 예시:

- "마지막으로 플레이어를 본 위치"
- "이미 수색한 방 목록"
- "동료 AI 의 상태"

예시:

- <팩맨>의 고스트: 플레이어의 마지막 위치를 기억하고 그 방향으로 이동
- <하프라이프>의 병사: 수류탄이 던져진 위치를 기억하고 회피

3. 목표 기반 에이전트 (Goal-based Agent)

명시적인 목표(Goal)를 가지고 그것을 달성하기 위해 행동을 계획합니다.

```
function GOAL_BASED_AGENT(percept):  
    state ← UPDATE_STATE(state, percept)  
  
    if NOT achieved(goal, state):  
        plan ← PLAN(state, goal)  
        action ← plan.NEXT_ACTION()  
    else:  
        goal ← SELECT_NEW_GOAL()  
  
    return action
```

목표 예시:

- "플레이어를 찾아라"
- "엄폐물 뒤에 숨어라"
- "체력을 회복해라"
- "깃발을 본진으로 가져와라"

예시:

- <심즈(The Sims)>: 배고픔, 피로, 사회성 등의 욕구를 목표로 행동 계획
- RTS 빌드 오더: "병영 3 개 건설 → 병사 10 명 생산 → 공격"

4. 효용 기반 에이전트 (Utility-based Agent)

여러 목표 중 효용(Utility) 점수가 가장 높은 것을 선택합니다.

```

function UTILITY_BASED_AGENT(percept):
    state ← UPDATE_STATE(state, percept)
    possibleActions ← GET_POSSIBLE_ACTIONS(state)

    bestAction ← NULL
    maxUtility ← -∞

    for each action in possibleActions do
        utility ← EVALUATE_UTILITY(action, state, utilityFunction)
        if utility > maxUtility then
            maxUtility ← utility
            bestAction ← action
        end if
    end for

    return bestAction

```

효용 함수 예시 (FPS AI):

행동	기본 효용	조건부 보정	최종 효용
공격	50	체력 < 30%: -30	20
엄폐	30	체력 < 30%: +40	70 ← 선택
재장전	20	탄약 < 20%: +50	70
후퇴	10	적 > 2 명: +30	40

AI 는 효용 점수가 가장 높은 '엄폐' 또는 '재장전'을 선택합니다.

예시:

- <심시티(SimCity)>: 교통, 범죄, 교육, 세금 등 복합 요소를 고려한 도시 관리
- <문명(Civilization)>: 외교, 경제, 군사력을 종합 평가하여 전략 선택

5. 학습 에이전트 (Learning Agent)

경험을 통해 성능을 개선하는 에이전트입니다.

```
function LEARNING_AGENT(percept):  
    state ← UPDATE_STATE(state, percept)  
  
    # 행동 선택  
    action ← PERFORMANCE_ELEMENT(state)  
  
    # 피드백 수집  
    feedback ← CRITIC(state, action, percept)  
  
    # 학습 (모델 업데이트)  
    LEARNING_ELEMENT(feedback)  
  
    return action
```

구성 요소:

- Performance Element: 현재 행동 선택
- Critic: 결과 평가 (보상/벌칙)
- Learning Element: 지식 업데이트
- Problem Generator: 탐색(Exploration) 유도

예시:

- 강화학습 기반 레이싱 AI: 코너링 실수 → 벌점 → 다음에는 속도 감소
- <포자(Spore)>: 생물 진화 시뮬레이션

1.3 게임 AI 설계 원칙

훌륭한 게임 AI 는 기술적 완성도뿐만 아니라 플레이어 경험을 고려한 설계 철학이 필요합니다.

1.3.1 적응형 난이도 조절 (Dynamic Difficulty Adjustment, DDA)

플레이어의 실력은 천차만별입니다. 고정된 난이도는 일부에게는 너무 쉽고, 일부에게는 너무 어렵습니다.

DDA 의 목표:

난이도 $\approx$ 플레이어 스킬 + ϵ

여기서 ϵ 은 작은 양수로, "약간의 도전"을 의미합니다.

DDA 구현 기법:

1. 명시적 DDA (Explicit DDA):

플레이어에게 난이도 선택권을 제공합니다.

난이도	적 체력	적 공격력	AI 반응 속도
쉬움	70%	70%	느림 (0.5 초 지연)
보통	100%	100%	보통 (0.3 초 지연)
어려움	150%	130%	빠름 (0.1 초 지연)

전문가	200%	150%	즉시 (지연 없음)
-----	------	------	------------

2. 암묵적 DDA (Implicit DDA):

플레이어 성과를 실시간 모니터링하여 자동 조절합니다.

```
function ADJUST_DIFFICULTY():
    if player.consecutiveDeaths >= 3:
        enemy.health *= 0.9
        enemy.damage *= 0.9
        showMessage("적이 약간 약해졌습니다")

    if player.consecutiveWins >= 5:
        enemy.health *= 1.1
        enemy.accuracy += 0.05
        showMessage("적이 강해졌습니다")
```

사례 연구: <레프트 포 데드(Left 4 Dead)>의 AI Director

- 긴장도(Intensity) 측정: 플레이어의 체력, 탄약, 적과의 거리를 종합 평가
- 피크 앤 밸리(Peaks and Valleys): 긴장-이완을 반복하여 리듬 생성
 - 긴장도 높음 → 좀비 스폰 감소, 회복 아이템 배치
 - 긴장도 낮음 → 특수 좀비(Tank, Witch) 스폰
- 결과: 매번 다른 경험, 높은 재플레이 가치

1.3.2 플레이어 경험과 AI 균형

공정성의 원칙:

AI 는 플레이어와 동일한 규칙을 따라야 합니다. 치트(Cheat)는 플레이어의 신뢰를 무너뜨립니다.

좋은 예:

- AI 도 플레이어와 같은 FOV 제한
- AI 도 재장전 시간 필요
- AI 도 장애물에 부딪히면 피해 받음

나쁜 예 (피해야 할 설계):

- AI 가 벽 너머 플레이어 위치를 정확히 알고 선제 공격
- AI 의 총알이 벽을 관통
- AI 의 명중률이 거리와 무관하게 100%

단, 전략 게임에서는 치트 AI 가 허용될 수 있습니다:

- 플레이어: "보통" 난이도
- AI: 자원 +50% 보너스 (하지만 명시됨)

핵심은 투명성(Transparency)입니다. 플레이어가 AI 의 이점을 알고 있다면 불공정하다고 느끼지 않습니다.

1.3.3 공정성과 예측 가능성

예측 가능성의 균형:

- 너무 예측 가능: 플레이어가 패턴을 파악하고 쉽게 악용
- 너무 랜덤: 플레이어가 전략을 세울 수 없어 좌절

예시: <다크 소울(Dark Souls)> 보스 AI

- 공격 패턴은 명확히 정해져 있음 (예측 가능)
- 공격 타이밍에 약간의 랜덤성 추가 (± 0.2 초)
- 플레이어는 패턴을 학습하면서 점진적으로 숙달

의외성(Surprise)의 활용:

플레이어가 패턴에 익숙해지면, 가끔 새로운 공격을 섞어 긴장감 유지.

```
if player.hasLearnedPattern():  
    if random() < 0.1: # 10% 확률  
        useNewAttackPattern()
```

1.3.4 AI 디버깅과 시각화

게임 AI 는 눈에 보이지 않는 로직으로 작동하므로 디버깅이 어렵습니다.

시각화 기법:

1. 상태 표시 (State Display):

```
void OnDrawGizmos()
{
    // 현재 상태 텍스트로 표시
    Handles.Label(transform.position + Vector3.up * 2,
        $"State: {currentState}\nTarget: {target?.name}");
}
```

2. 센서 범위 표시:

```
void OnDrawGizmosSelected()
{
    // 시야각 표시 (부채꼴)
    Gizmos.color = Color.yellow;
    Vector3 forward = transform.forward * viewRadius;
    Vector3 rightBoundary = Quaternion.Euler(0, viewAngle / 2, 0) * forward;
    Vector3 leftBoundary = Quaternion.Euler(0, -viewAngle / 2, 0) * forward;

    Gizmos.DrawLine(transform.position, transform.position + rightBoundary);
    Gizmos.DrawLine(transform.position, transform.position + leftBoundary);
    Gizmos.DrawRay(transform.position, forward);
}
```

3. 경로 표시:

```
void OnDrawGizmos()
{
    if (path != null)
    {
        for (int i = 0; i < path.Count - 1; i++)
        {
```

```
        Gizmos.color = Color.blue;
        Gizmos.DrawLine(path[i], path[i + 1]);
    }
}
```

수학적 분석

정리 1.1 (에이전트의 최적성)

에이전트의 성능 측정 함수를 $P(h)$ 라 할 때 (h 는 에이전트의 행동 이력), 합리적 에이전트(Rational Agent)는 기댓값 $E[P(h)]$ 를 최대화하는 에이전트입니다.

$$\text{Agent}^* = \arg \max E[P(h)]$$

증명 (스케치):

- 1. 에이전트는 각 시점 t 에서 지각 p_t 를 받고 행동 a_t 를 선택합니다.
- 2. 행동 이력: $h = (p_1, a_1, p_2, a_2, \dots, p_n, a_n)$
- 3. 성능 측정: $P(h) = \sum r_t$ (보상의 합)
- 4. 최적 에이전트는 모든 가능한 행동 시퀀스 중 기댓값이 최대인 것을 선택합니다.

하지만 게임 AI 에서는 $P(h)$ 가 단순히 "승리"가 아니라 "플레이어 만족도"를 포함해야 하므로, 수학적 최적성과 실제 목표가 다릅니다. □

정리 1.2 (효용 함수의 선형 결합)

효용 기반 에이전트에서 여러 요소의 효용을 결합할 때, 선형 결합을 사용합니다:

$$U(\text{action}) = \sum w_i \cdot u_i(\text{action})$$

여기서:

- u_i : i 번째 효용 요소 (예: 체력, 탄약, 전술적 위치)
- w_i : i 번째 요소의 가중치 ($\sum w_i = 1$)

예제:

$$\begin{aligned} U(\text{공격}) &= 0.4 \times u_{\text{damage}}(\text{공격}) + 0.3 \times u_{\text{safety}}(\text{공격}) + 0.3 \times u_{\text{ammo}}(\text{공격}) \\ &= 0.4 \times 0.8 + 0.3 \times 0.3 + 0.3 \times 0.6 \\ &= 0.32 + 0.09 + 0.18 \\ &= 0.59 \end{aligned}$$

가중치 w_i 의 조정으로 AI의 성격(공격적/방어적)을 제어할 수 있습니다. □

의사코드

알고리즘 1.1: 단순 반사 에이전트

```
SIMPLE_REFLEX_AGENT(percept, rules)
  state ← INTERPRET_INPUT(percept)

  for each rule in rules do
    if MATCH(rule.condition, state) then
      return rule.action
    end if
  end for

  return DEFAULT_ACTION
```

알고리즘 1.2: 효용 기반 에이전트

```
UTILITY_BASED_AGENT(percept, state, utilityFunction)
  state ← UPDATE_STATE(state, percept)
  possibleActions ← GET_POSSIBLE_ACTIONS(state)

  bestAction ← NULL
  maxUtility ←  $-\infty$ 

  for each action in possibleActions do
    utility ← EVALUATE_UTILITY(action, state, utilityFunction)
    if utility > maxUtility then
      maxUtility ← utility
      bestAction ← action
    end if
  end for

  return bestAction

EVALUATE_UTILITY(action, state, utilityFunction)
  totalUtility ← 0

  for each factor in utilityFunction.factors do
    value ← factor.EVALUATE(action, state)
    totalUtility ← totalUtility + factor.weight × value
  end for

  return totalUtility
```

알고리즘 1.3: 적응형 난이도 조절 (DDA)

```
DYNAMIC_DIFFICULTY_ADJUSTMENT(playerStats, gameParams)
# 플레이어 성과 추적
if playerStats.consecutiveDeaths >= threshold_death then
    # 난이도 하향
    gameParams.enemyHealth ← gameParams.enemyHealth × 0.9
    gameParams.enemyDamage ← gameParams.enemyDamage × 0.9
    playerStats.consecutiveDeaths ← 0
    SHOW_MESSAGE("난이도가 조정되었습니다")
end if

if playerStats.consecutiveWins >= threshold_win then
    # 난이도 상향
    gameParams.enemyHealth ← gameParams.enemyHealth × 1.1
    gameParams.enemyAccuracy ← MIN(gameParams.enemyAccuracy + 0.05, 0.95)
    playerStats.consecutiveWins ← 0
end if

# 점진적 복귀 (시간이 지나면 기본값으로)
gameParams.enemyHealth ← LERP(gameParams.enemyHealth, baseHealth, 0.01)
gameParams.enemyDamage ← LERP(gameParams.enemyDamage, baseDamage, 0.01)

return gameParams
```

요약

핵심 개념

- 게임 AI vs 학술적 AI: 게임 AI 는 최적성보다 재미와 플레이어 경험을 우선합니다.
- 믿을 만함(Believability): 완벽한 AI 보다 인간적인 실수를 하는 AI 가 더 좋은 경험을 제공합니다.
- PEAS 모델: Performance, Environment, Actuators, Sensors 로 에이전트를 체계적으로 설계합니다.
- 에이전트 유형: 단순 반사 → 모델 기반 → 목표 기반 → 효용 기반 → 학습 순으로 복잡도가 증가합니다.
- 효용 함수: 여러 목표의 가중치 합으로 복잡한 의사결정을 구현합니다.
- DDA (적응형 난이도 조절): 플레이어 실력에 맞춰 난이도를 동적으로 조절하여 몰입 상태를 유지합니다.
- 계층적 아키텍처: 전략-전술-행동-반사 계층으로 복잡한 AI 를 구조화합니다.

언제 쓰고/언제 피하는가

사용하면 좋은 경우:

- 플레이어와 상호작용하는 NPC 설계 시 PEAS 모델 활용
- 복잡한 의사결정이 필요할 때 효용 기반 에이전트 사용
- 다양한 플레이어 실력 수준을 대응할 때 DDA 적용
- AI 디버깅이 어려울 때 시각화 기법 활용

피해야 할 경우:

- 단순한 행동에 과도하게 복잡한 모델 적용 (오버엔지니어링)
- 플레이어가 인지할 수 없는 미세한 AI 최적화
- 투명하지 않은 치트 AI 설계 (플레이어 신뢰 저하)

- 효용 함수의 가중치를 임의로 설정 (플레이테스트 필수)

연습문제 (10 문제)

기초 문제 (1-4)

1. [개념] 게임 AI 와 학술적 AI 의 가장 큰 차이점은 무엇인가? '재미'와 '최적성'의 관점에서 서술하시오.
2. [튜링 테스트] 게임 AI 에서 "믿을 만함(Believability)"이 "완벽한 성능"보다 중요한 이유를 FPS 저격수 AI 의 예를 들어 설명하시오.
3. [PEAS 모델] 레이싱 게임의 상대 AI 를 PEAS 모델로 설계하시오. Performance, Environment, Actuators, Sensors 각각을 구체적으로 기술하시오.
4. [에이전트 유형] 다음 게임 AI 를 에이전트 유형(단순 반사, 모델 기반 반사, 목표 기반, 효용 기반, 학습)으로 분류하고 그 이유를 설명하시오:
 - (a) <스페이스 인베이더>의 외계인
 - (b) <팩맨>의 고스트
 - (c) <심즈>의 캐릭터
 - (d) 강화학습 기반 <도타 2> AI

응용 문제 (5-7)

5. [효용 함수] FPS AI 가 다음 3 가지 행동 중 하나를 선택합니다. 효용 함수 $U(\text{action}) = 0.5 \times \text{damage} + 0.3 \times \text{safety} + 0.2 \times \text{ammo}$ 를 사용하여 최선의 행동을 계산하시오.

행동	damage	safety	ammo
공격	0.8	0.3	0.4
엄폐	0.1	0.9	0.8
재장전	0.0	0.6	1.0

6. [DDA 설계] 플레이어가 3 번 연속 사망했을 때, 적의 체력과 공격력을 각각 10%

감소시키는 DDA 시스템을 설계하시오. 연속 승리 시 난이도를 어떻게 조정할지도 함께 기술하시오.

7. [공정성] 다음 중 플레이어에게 불공정하게 느껴질 수 있는 AI 설계를 모두 고르고,

각각을 공정하게 개선하는 방법을 제시하시오:

- (a) AI 가 벽 너머 플레이어 위치를 정확히 알고 선제 공격
- (b) AI 의 시야각이 120°로 제한됨
- (c) AI 의 반응 속도가 0.3 초 지연
- (d) AI 의 총알이 벽을 관통

Chapter 2. 유한 상태 기계 (Finite State Machine, FSM)

FSM 은 게임 AI 를 구현하는 가장 고전적이면서도 여전히 강력한 기법입니다. 1980 년대 <팩맨>부터 현대의 AAA 타이틀까지, FSM 은 NPC 행동을 제어하는 핵심 도구로 자리잡았습니다. 본 챕터에서는 FSM 의 이론적 기초부터 실전 구현, 그리고 복잡도 관리를 위한 계층적 설계까지 다룹니다.

2.1 오토마타 이론과 상태 공간

2.1.1 오토마타 정의

정의 2.1 (유한 오토마타)

유한 오토마타(Finite Automaton)는 다음 5 개 요소로 정의됩니다:

$$M = (Q, \Sigma, \delta, q_0, F)$$

여기서:

- Q : 유한한 상태(State)의 집합
- Σ : 입력 알파벳(Input Alphabet)의 집합
- $\delta: Q \times \Sigma \rightarrow Q$, 전이 함수(Transition Function)
- $q_0 \in Q$: 초기 상태(Initial State)
- $F \subseteq Q$: 최종 상태(Accepting States)의 집합

게임 AI에서는 F(최종 상태)가 없는 무한 루프 오토마타를 주로 사용합니다. 게임이 종료될 때까지 계속 실행되기 때문입니다.

2.1.2 결정론적 vs 비결정론적 유한 오토마타

결정론적 유한 오토마타 (Deterministic Finite Automaton, DFA):

각 상태에서 각 입력에 대해 정확히 하나의 다음 상태가 결정됩니다.

```
 $\delta(\text{Patrol}, \text{PlayerSpotted}) = \text{Chase}$  // 명확히 하나의 전이
```

비결정론적 유한 오토마타 (Nondeterministic Finite Automaton, NFA):

동일한 입력에 대해 여러 개의 가능한 전이가 있을 수 있습니다.

```
 $\delta(\text{Combat}, \text{LowHealth}) = \{\text{Retreat}, \text{UseHealthPotion}, \text{CallForBackup}\}$ 
```

게임 AI에서는 주로 DFA를 사용하지만, 랜덤성이 필요한 경우 NFA를 활용합니다:

```
State NextState(State current, Input input)
{
    List<State> possibleStates = nfa[current][input];
    return possibleStates[Random.Range(0, possibleStates.Count)];
}
```

2.1.3 상태 전이 함수

전이 함수 δ 는 게임 AI 의 두뇌입니다.

예제 2.1: 경비병 AI 의 전이 함수

현재 상태 (q)	입력 (σ)	다음 상태 $\delta(q, \sigma)$
Patrol	PlayerSpotted	Chase
Patrol	SuspiciousSound	Investigate
Patrol	-	Patrol
Chase	InAttackRange	Attack
Chase	LostPlayer	Search
Chase	-	Chase
Attack	PlayerDead	Patrol
Attack	OutOfRange	Chase
Attack	LowHealth	Retreat
Investigate	PlayerFound	Chase
Investigate	NoThreat	Patrol
Search	TimeOut	Patrol

의사코드 표현:

```
 $\delta(\text{Patrol}, \text{PlayerSpotted}) \rightarrow \text{Chase}$   
 $\delta(\text{Chase}, \text{InAttackRange}) \rightarrow \text{Attack}$   
 $\delta(\text{Attack}, \text{LowHealth}) \rightarrow \text{Retreat}$   
...
```

2.1.4 게임 AI의 상태 표현

상태는 단순한 레이블이 아니라 행동(Behavior)을 캡슐화합니다.

상태의 3 가지 메서드:

```
public interface IState
{
    void Enter();    // 상태 진입 시 1 회 실행
    void Update();   // 매 프레임 실행
    void Exit();     // 상태 이탈 시 1 회 실행
}
```

예제 2.2: Patrol 상태 구현 (일부)

```
public class PatrolState : IState
{
    private AIController ai;
    private int currentWaypoint = 0;

    public void Enter()
    {
        Debug.Log("순찰 시작");
        ai.SetSpeed(2.0f);
        ai.SetAnimation("Walk");
    }

    public void Update()
    {
        // 웨이포인트 따라가기
        if (ai.ReachedWaypoint())
        {
            currentWaypoint = (currentWaypoint + 1) % ai.waypoints.Count;
        }
        ai.MoveToward(ai.waypoints[currentWaypoint]);

        // 플레이어 감지 체크 (전이 조건)
        if (ai.CanSeePlayer())
        {

```

```
        ai.ChangeState(new ChaseState(ai));
    }
}

public void Exit()
{
    Debug.Log("순찰 종료");
    ai.StopMoving();
}
}
```

2.2 FSM 설계와 구현

2.2.1 FSM 구성요소

FSM 은 다음 3 가지 핵심 요소로 구성됩니다:

1. 상태 (State)

캐릭터가 수행 중인 행동이나 상황:

- 대기 (Idle)
- 순찰 (Patrol)
- 추격 (Chase)
- 공격 (Attack)
- 도망 (Retreat)
- 수색 (Search)
- 죽음 (Death)

2. 전이 (Transition)

한 상태에서 다른 상태로 바뀌는 흐름:

- 단방향 화살표로 표현
- 조건이 만족될 때 실행
- 원자적(Atomic) - 즉시 완료

3. 조건 (Condition/Input)

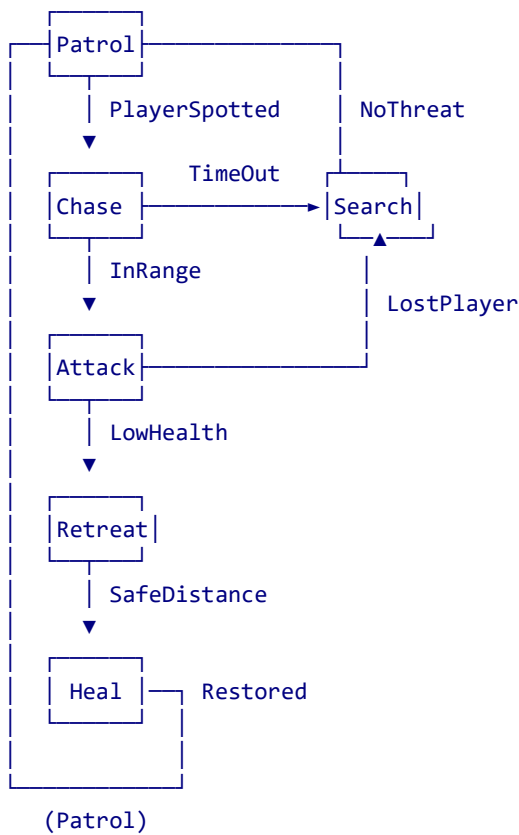
전이를 유발하는 이벤트:

- 센서 입력: "적 발견", "소리 들림"
- 내부 상태: "체력 < 30%", "탄약 소진"
- 타이머: "10 초 경과", "애니메이션 종료"
- 게임 이벤트: "플레이어 사망", "미션 완료"

2.2.2 상태 전이 다이어그램 (State Transition Diagram)

FSM 은 그래프 형태로 시각화됩니다.

예제 2.3: 경비병 AI 상태 다이어그램 (텍스트 표현)



다이어그램 작성 규칙:

- 1. 원/사각형: 상태
- 2. 화살표: 전이
- 3. 화살표 위 텍스트: 조건

- 4. 이중 원: 초기 상태
- 5. 굵은 테두리: 주요 상태

2.2.3 구현 방법 1: Switch-Case

가장 단순한 형태의 구현입니다.

```
public class AIController : MonoBehaviour
{
    public enum State { Patrol, Chase, Attack, Retreat }
    private State currentState = State.Patrol;

    void Update()
    {
        switch (currentState)
        {
            case State.Patrol:
                Patrol();
                break;
            case State.Chase:
                Chase();
                break;
            case State.Attack:
                Attack();
                break;
            case State.Retreat:
                Retreat();
                break;
        }
    }

    void Patrol()
    {
        // 순찰 로직
        MoveAlongPatrolPath();
        if (CanSeePlayer())
        {
            currentState = State.Chase;
        }
    }

    void Chase()
    {
        // 추격 로직
        MoveToward(player.position);
        if (DistanceToPlayer() < attackRange)
        {
            currentState = State.Attack;
        }
    }
}
```

```
        currentState = State.Attack;
    }
    else if (!CanSeePlayer())
    {
        currentState = State.Patrol;
    }
}
}
```

장점:

- 구현 매우 간단
- 디버깅 쉬움 (한 곳에서 모든 로직 확인)
- 성능 우수 (함수 호출 오버헤드 없음)

단점:

- 상태가 많아지면 관리 어려움 (스파게티 코드)
- Enter/Exit 로직 구현 번거로움
- 재사용성 낮음
- 확장성 부족

2.2.4 구현 방법 2: State Pattern (추천)

객체지향 디자인 패턴을 활용한 구현입니다.

IState 인터페이스:

```
public interface IState
{
    void Enter();
    void Update();
    void Exit();
}
```

AI Controller (상태 관리자):

```
public class AIController : MonoBehaviour
{
    private IState currentState;

    void Start()
    {
        ChangeState(new PatrolState(this));
    }

    void Update()
    {
        currentState?.Update();
    }

    public void ChangeState(IState newState)
    {
        currentState?.Exit();
        currentState = newState;
        currentState?.Enter();
    }
}
```

장점:

- 각 상태가 독립적 (캡슐화)
- Enter/Exit 명확히 구분
- 재사용 가능
- 확장 용이 (새 상태 추가 시 기존 코드 수정 불필요)
- 팀 협업에 유리

단점:

- 초기 구조 설계 필요
- 클래스 파일 많아짐 (관리 필요)
- Switch-Case 보다 약간 느림 (실무에서 무시 가능)

2.3 계층적 FSM (HFSM)

2.3.1 HFSM의 필요성

단순 FSM의 문제점:

상태 폭발 (State Explosion):

게임이 복잡해질수록 상태 수가 기하급수적으로 증가합니다.

예제 2.4: RPG 몬스터 AI

- Idle
- Patrol
- Chase
- MeleeAttack
- RangedAttack
- CastSpell_Fireball
- CastSpell_IceBolt
- CastSpell_Heal
- Dodge
- Block
- Retreat
- Eat (체력 회복)
- Sleep (대기)
- CallAlly
- ...

15 개 상태 $\times$ 15 개 상태 = 225 개의 가능한 전이!

전이 선 복잡도:

모든 상태가 서로 연결되면:

최대 전이 수 = $n \times (n - 1) = O(n^2)$

- 10 개 상태: 90 개 전이
- 20 개 상태: 380 개 전이

이는 유지보수 불가능한 "스파게티 코드"를 만듭니다.

2.3.2 HFSM 의 구조

정의 2.2 (계층적 FSM)

HFSM 은 상태가 또 다른 FSM 을 포함할 수 있는 중첩 구조입니다. 상위 상태(Super-state)는 여러 하위 상태(Sub-state)를 가집니다.

예제 2.5: 계층적 전투 상태 (텍스트 표현)



외부에서는 [Combat] 상태로 진입하지만, 내부적으로는 거리에 따라 [CloseRange] 또는 [LongRange]로 분기합니다.

2.3.4 복잡도 관리

복잡도 비교:

구조	상태 수	전이 수 (최악)	관리 난이도
단순 FSM	n	$n(n-1) = O(n^2)$	높음
2 계층 HFSM	$n_1 \times n_2$	$n_1^2 + n_2^2 = O(n)$	중간

3 계층 HFSM	$n_1 \times n_2 \times n_3$	$n_1^2 + n_2^2 + n_3^2 =$ $O(n)$	낮음
-----------	-----------------------------	-------------------------------------	----

예시:

- 단순 FSM: 20 개 상태 → 380 개 전이
- HFSM:
 - - 상위 4 개 상태 → 12 개 전이
 - - 각 상위당 평균 5 개 하위 → $4 \times 20 = 80$ 개 전이
 - - 총합: 92 개 전이 (76% 감소!)

2.4 상용 게임 사례 분석

2.4.1 팩맨 고스트 AI

<팩맨>(1980)의 고스트 AI 는 FSM 의 고전적 사례입니다.

4 가지 상태:

- 1. Chase (추적): 팩맨을 쫓습니다.
 - Blinky (빨강): 팩맨을 직접 추격
 - Pinky (분홍): 팩맨 앞쪽 4 칸 목표
 - Inky (청록): Blinky 와 협동 위치
 - Clyde (주황): 팩맨 가까우면 도망, 멀면 추격
- 2. Scatter (분산): 각자의 코너로 흩어집니다.
- 3. Frightened (겁먹음): 파워 펠릿 먹으면 랜덤 이동합니다.
- 4. Eaten (복귀): 잡히면 눈만 남아 집으로 돌아갑니다.

전이 조건:

- Chase ↔ Scatter: 7 초마다 자동 전환 (게임 시작 20 초 후부터)
- Chase/Scatter → Frightened: 파워 펠릿 섭취
- Frightened → Chase/Scatter: 10 초 경과 또는 팩맨 접촉
- Any → Eaten: 겁먹은 상태에서 팩맨 접촉
- Eaten → Chase: 집 도착

놀라운 점: 단 4 개 상태로 복잡한 협동 행동 구현!

요약

핵심 개념

- 오토마타 이론: FSM 은 5-튜플 $(Q, \Sigma, \delta, q_0, F)$ 로 정의됩니다.
- DFA vs NFA: DFA 는 결정론적, NFA 는 비결정론적 전이를 허용합니다.
- 상태 3 요소: 상태(State), 전이(Transition), 조건(Condition)으로 구성됩니다.
- 구현 방법: Switch-Case (단순), State Pattern (확장성), Table-Driven (데이터 주도).
- 상태 폭발: 평탄 FSM 은 $O(n^2)$ 복잡도로 관리 불가능해집니다.
- HFSM: 계층적 구조로 복잡도를 $O(k \cdot m^2)$ 로 감소시킵니다.
- 실제 사례: 팩맨, 스타크래프트, 하프라이프 등 다양한 게임에서 검증된 기법입니다.

언제 쓰고/언제 피하는가

사용하면 좋은 경우:

- 명확히 구분되는 행동 상태가 있을 때
- 상태 전이 규칙이 명확할 때
- 디버깅과 시각화가 중요할 때
- 팀 협업에서 명확한 설계가 필요할 때
- 단순하고 예측 가능한 AI 가 필요할 때

피해야 할 경우:

- 상태가 8 개 이상일 때 (HFSM 사용 권장)
- 복잡한 조건 판단이 필요할 때 (BT, GOAP 고려)
- 동적이고 창의적인 AI 가 필요할 때
- 상태 간 경계가 모호할 때
- 다수의 동시 행동이 필요할 때

참고 자료

교재

1. Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson. (Chapter 2: Intelligent Agents)
2. Millington, I., & Funge, J. (2009). Artificial Intelligence for Games (2nd ed.). CRC Press.
3. Buckland, M. (2005). Programming Game AI by Example. Wordware Publishing.

논문

1. Yannakakis, G. N., & Togelius, J. (2018). "Artificial Intelligence and Games". Springer.
2. Hunicke, R. (2005). "The Case for Dynamic Difficulty Adjustment in Games." ACE Conference.

게임 AI 사례

1. Valve. (2009). "The AI Systems of Left 4 Dead." Valve Developer Community.
2. Monolith. (2006). "The AI of FEAR: Three Techniques for Advanced AI." Game Developers Conference.
3. Bungie. (2002). "The Integration of AI and Level Design in Halo." GDC.

Part II. 환경 인지와 불확실성

게임 인공지능

NPC의 움직임과 판단, 학습을 만드는 13 가지 알고리즘 패턴

Part II 개요

Part I에서는 유한 상태 기계(FSM)를 중심으로 에이전트의 기본 구조와 의사결정 메커니즘을 다뤘습니다. 하지만 FSM 만으로는 "적이 어떻게 플레이어를 인식하는가?", "왜 어떤 게임의 AI는 공정하고 어떤 AI는 치트처럼 느껴지는가?", "어떻게 모든 플레이어에게 적절한 난이도를 제공할 수 있는가?"와 같은 질문에 답할 수 없습니다.

Part II에서는 게임 AI가 불완전한 정보 속에서 합리적으로 판단하고, 공정한 센서 시스템을 통해 세상을 인식하며, 확률적 메커니즘으로 예측 불가능성과 재미를 만들어내는 방법을 학습합니다. Warcraft 3, Dota 2, The Last of Us 등 AAA 게임의 실제 기법을 수학적으로 모델링하고 Unity C#로 구현합니다.

학습 목표

본 Part 를 학습한 후 여러분은 다음을 수행할 수 있어야 합니다:

1. 확률론과 베이즈 정리를 활용하여 불완전한 정보 상황을 수학적으로 모델링할 수 있다.
2. 확률 분포의 개념을 이해하고 게임 밸런싱에 적용할 수 있다.
3. 가중치 랜덤과 PRD(Pseudo-Random Distribution) 시스템을 Unity C#로 구현할 수 있다.
4. 시각 및 청각 센서를 수학적으로 모델링하고 구현할 수 있다.
5. 다중 센서 융합과 메모리 시스템을 통해 AAA 급 지각 시스템을 구축할 수 있다.
6. 폴링과 이벤트 방식의 성능 차이를 분석하고 최적화할 수 있다.

Part II 구조

Chapter	제목	주요 내용	분량
3	확률론과 게임 AI 의 불확실성	확률 공간, 베이즈 정리, 확률 분포, 가중치 랜덤, PRD, DDA	~35p
4	센서와 지각 시스템	센서 모델링, 시각/청각 시스템, 센서 융합, 메모리, 최적화	~35p

Chapter 3. 확률론과 게임 AI 의 불확실성

3.0 도입과 동기

왜 게임에 확률이 필요한가?

문제 1: 극단적인 운의 연속

RPG 게임에서 플레이어의 치명타 확률이 25%라고 가정해봅시다. 이론적으로 4 번 공격하면 평균 1 번 치명타가 발생해야 합니다. 하지만 순수한 난수를 사용하면 15 번 연속 공격해도 치명타가 한 번도 안 나올 확률이 약 1.3%나 됩니다. 100 명의 플레이어 중 1 명은 이런 "불운"을 경험하게 되며, 그 플레이어는 "확률이 거짓말이다"라고 느낄 것입니다.

문제 2: 불공정한 AI 추론

RTS 게임에서 정찰병이 북쪽에서 적 유닛 1 기를 발견했습니다. 만약 AI 가 "적 본진이 북쪽에 있다"고 100% 확신한다면 부자연스럽습니다. 하지만 확률적 추론 없이는 AI 는 0% 또는 100%의 극단적 판단만 할 수 있습니다.

문제 3: 획일적인 난이도

모든 플레이어에게 동일한 적 체력과 공격력을 제공하면, 초보자는 "너무 어렵다"며 포기하고 고수는 "너무 쉽다"며 지루해합니다.

해결책: 확률론

이 장에서는 베이지 정리로 불완전한 정보를 추론하고, PRD 로 감정적 공정성을 제공하며,
DDA(Dynamic Difficulty Adjustment)로 플레이어 실력에 맞는 난이도를 동적으로
조절하는 방법을 학습합니다.

3.1 확률 이론 기초와 베이즈 정리

3.1.1 확률 공간 (Probability Space)

정의 3.1 (확률 공간)

확률 공간(Probability Space)은 3 개의 요소로 구성된 튜플 $(\Omega, \mathcal{F}, P)$ 입니다:

- Ω (표본 공간, Sample Space): 모든 가능한 결과의 집합
- $\mathcal{F}$ (사건 공간, Event Space): Ω 의 부분집합들로 이루어진 σ -대수
- P (확률 측도, Probability Measure): 각 사건에 $[0,1]$ 범위의 확률을 부여하는 함수

예제 3.1: RPG 전투 시스템

공격 결과의 확률 공간:

- $\Omega = \{\text{Hit}, \text{Miss}, \text{Critical}, \text{Dodge}\}$
- $P(\text{Hit}) = 0.70, P(\text{Miss}) = 0.15, P(\text{Critical}) = 0.10, P(\text{Dodge}) = 0.05$
- $P(\Omega) = 1.00$ (완전성)

3.1.2 조건부 확률 (Conditional Probability)

정의 3.2 (조건부 확률)

사건 B 가 발생했을 때 사건 A 가 발생할 확률:

$$P(A/B) = P(A \cap B) / P(B), \quad P(B) > 0$$

게임 적용: FPS 전술 AI

$P(\text{HP} \leq 30\% \mid \text{엄폐 중})$ = AI 가 엄폐 중일 때 체력이 30% 이하일 확률

이 확률이 높으면 AI 는 다음 행동을 선택합니다:

- 수류탄 투척
- 측면 우회
- 집중 사격

3.1.3 베이즈 정리 (Bayes' Theorem)

정리 3.1 (베이즈 정리)

$$P(A/B) = [P(B/A) \times P(A)] / P(B)$$

구성 요소:

- $P(A|B)$: 사후 확률 (Posterior) - 증거 B 를 관찰한 후 A 의 확률
- $P(B|A)$: 우도 (Likelihood) - A 가 참일 때 B 가 관찰될 확률
- $P(A)$: 사전 확률 (Prior) - 증거 전의 A 확률
- $P(B)$: 증거 확률 (Evidence) - 정규화 상수

예제 3.2: RTS 게임 - 적 본진 위치 추론

변수	의미	확률
A	적 본진이 북쪽에 있다	$P(A) = 0.25$
B	정찰병이 북쪽에서 적 발견	$P(B A) = 0.80$
$\neg A$	적 본진이 다른 곳에 있다	$P(B \neg A) = 0.10$

계산:

$$\begin{aligned} P(A/B) &= [0.80 \times 0.25] / [0.80 \times 0.25 + 0.10 \times 0.75] \\ &= 0.20 / 0.275 \approx 0.727 \end{aligned}$$

결과: 사전 확률 25% → 관찰 후 사후 확률 72.7%

AI 는 북쪽 정찰 강화 또는 방어 태세 전환을 결정합니다.

3.1.4 전확률 공식 (Law of Total Probability)

정리 3.2 (전확률 공식)

$$P(A) = \sum P(A/B_i) \times P(B_i)$$

예제 3.3: 날씨별 명중률 계산

날씨	사전 확률	명중률	기여도
맑음	0.60	0.85	0.51
비	0.30	0.65	0.195
안개	0.10	0.40	0.04

전체 명중률: $P(\text{명중}) = 0.51 + 0.195 + 0.04 = 0.745$ (74.5%)

3.2 확률 분포와 게임 밸런싱

3.2.1 이산 확률 분포 (Discrete Probability Distributions)

균등 분포 (Uniform Distribution)

$$P(X = x_i) = 1/n, \quad i = 1, 2, \dots, n$$

응용:

- 주사위 굴림: $P(1) = P(2) = \dots = P(6) = 1/6$
- 무작위 스폰 위치 선택
- 공정한 카드 뽑기

한계: 현실적인 게임플레이에는 부자연스러움

이항 분포 (Binomial Distribution)

$$P(X = k) = C(n, k) \times p^k \times (1-p)^{n-k}$$

예제 3.4: 다중 공격 시스템

캐릭터가 5 번 공격, 각 공격 명중률 70%일 때 정확히 3 번 맞출 확률:

$$P(X=3) = C(5,3) \times (0.7)^3 \times (0.3)^2 = 10 \times 0.343 \times 0.09 \approx 0.3087$$

결과: 약 30.87% 확률

3.2.2 연속 확률 분포 (Continuous Probability Distributions)

정규 분포 (Normal Distribution)

$$f(x) = [1/(\sigma\sqrt{2\pi})] \times e^{-((x-\mu)^2/(2\sigma^2))}$$

게임 적용 사례:

시스템	평균 μ	표준편차 σ	설명
NPC 키	175cm	10cm	대부분 165-185cm
이동속도	1.0	0.1	$\pm 10\%$ 변동
데미지 범위	100	15	70-130 범위 분포

지수 분포 (Exponential Distribution)

$$f(t) = \lambda \times e^{-\lambda t}, \quad t \geq 0$$

예제 3.5: 적 스폰 타이밍

평균 30 초마다 스폰 ($\lambda = 1/30$):

- $P(20 \text{ 초 이내 스폰}) = 1 - e^{-20/30} \approx 0.487 \text{ (48.7\%)}$
- $P(60 \text{ 초 이내 스폰}) = 1 - e^{-60/30} \approx 0.865 \text{ (86.5\%)}$

3.2.3 기댓값과 분산 (Expected Value and Variance)

정의 3.3: 기댓값

$$E[X] = \sum x_i \times P(X = x_i) \quad (\text{이산})$$

$$E[X] = \int x \cdot f(x) dx \quad (\text{연속})$$

정의 3.4: 분산

$$Var(X) = E[(X - E[X])^2] = E[X^2] - (E[X])^2$$

$$\text{표준편차 } \sigma = \sqrt{Var(X)}$$

예제 3.6: 아이템 드롭 기댓값

아이템	골드 가치	드롭 확률
포션	10	0.50
무기	100	0.30
방어구	150	0.15
전설템	1000	0.05

기댓값:

$$E[\text{골드}] = 10(0.50) + 100(0.30) + 150(0.15) + 1000(0.05) = 107.5$$

분산: $Var \approx 45044.38$, 표준편차 $\sigma \approx 212.2$

해석: 높은 분산 = 불안정한 보상 경험 → 밸런싱 필요

3.3 가중치 랜덤과 의사 난수

3.3.1 가중치 랜덤 선택 (Weighted Random Selection)

문제: 아이템 등급별로 다른 확률 부여 필요

알고리즘 3.1: CDF 방법 (누적 분포 함수)

```
Input: items[], weights[], n
Output: Selected item index

1. cumulativeWeights[0] = weights[0]
2. for i = 1 to n-1:
    cumulativeWeights[i] = cumulativeWeights[i-1] + weights[i]
3. totalWeight = cumulativeWeights[n-1]
4. randomValue = Random(0, totalWeight)
5. for i = 0 to n-1:
    if randomValue < cumulativeWeights[i]:
        return i
```

시간 복잡도: $O(n)$ 초기화, $O(n)$ 선택

최적화: 이진 탐색

```
6. left = 0, right = n-1
7. while left < right:
8.     mid = (left + right) / 2
9.     if randomValue < cumulativeWeights[mid]:
10.         right = mid
11.     else:
12.         left = mid + 1
13. return left
```

최적화 복잡도: $O(n)$ 초기화, $O(\log n)$ 선택

예제 3.7: 아이템 드롭 테이블

등급	가중치	누적	확률
Common	500	500	50.0%
Rare	300	800	30.0%
Epic	150	950	15.0%
Heroic	40	990	4.0%
Legendary	10	1000	1.0%

Unity C# 구현:

```
public class WeightedRandomSelector
{
    private List<float> cumulative;
    private float total;

    public void Initialize(float[] weights)
    {
        cumulative = new List<float>();
        total = 0f;
        foreach (float weight in weights)
        {
            total += weight;
            cumulative.Add(total);
        }
    }

    public int SelectIndex()
    {
        float randVal = Random.Range(0f, total);
        int left = 0;
        int right = cumulative.Count - 1;

        while (left < right)
        {
            int mid = (left + right) / 2;
            if (randVal < cumulative[mid])
                right = mid;
            else
                left = mid + 1;
        }
        return left;
    }
}
```

3.3.2 Alias Method

대규모 샘플링이 필요한 경우 $O(1)$ 선택 시간을 제공하는 방법

방법	전처리	샘플링	메모리	적합한 경우
CDF (선형)	$O(n)$	$O(n)$	$O(n)$	작은 n , 드문 샘플링
CDF (이진)	$O(n)$	$O(\log n)$	$O(n)$	중간 n
Alias	$O(n)$	$O(1)$	$O(n)$	대규모 샘플링

응용: 파티클 시스템, 실시간 몬스터 스폰, 절차적 생성

3.3.3 의사 난수 분포 (Pseudo-Random Distribution, PRD)

순수 난수의 문제

20% 치명타 확률로 10 번 연속 공격 시 한 번도 발생하지 않을 확률:

$$P(10 \text{ 연속 실패}) = (0.80)^{10} \approx 0.107 = 10.7\%$$

결과: 10 명 중 1 명은 "불운" 경험 → 확률이 조작된 것처럼 느낌

PRD 원리 (Warcraft 3, Dota 2 사용)

- 1. 낮은 확률 P_C 로 시작 ($<$ 목표 확률 P)
- 2. 실패 시 확률을 ΔP 만큼 증가
- 3. 성공 시 P_C 로 초기화
- 4. 장기적으로 실제 확률은 P 로 수렴

예제 3.8: 20% 목표에 대한 PRD 파라미터 계산

시도	누적 실패(순수)	PRD 확률	PRD 누적 실패
1	0.80	0.038	0.962
2	0.64	0.076	0.889
3	0.512	0.114	0.788
5	0.328	0.190	0.551
10	0.107	0.380	0.108

결과: 10 연속 실패가 10.7%(순수) → 10.8%(PRD)로 비슷하지만, 중간 시도들의 분산이 크게 감소

수학적 모델:

$$P = \sum_{(n=1 \rightarrow \infty)} n \times P_C \times (1-P_C)^{(n-1)}$$

C# 구현:

```
public class PRDSystem
{
    private float initialProbability;
    private float increment;
    private float currentProbability;

    public PRDSystem(float targetProbability)
    {
        // 근사식: P_C ≈ 0.019 × P
        this.initialProbability = 0.019f * targetProbability;
        this.increment = initialProbability;
        this.currentProbability = initialProbability;
    }

    public bool TryProc()
    {
        bool success = Random.value < currentProbability;
        if (success)
        {
            currentProbability = initialProbability; // 초기화
        }
        else
        {
            currentProbability += increment; // 증가
        }
        return success;
    }
}
```

장점:

- 불운한 연속 실패를 체계적으로 방지
- 실제 확률은 빠르게 수렴
- 예측 가능성과 무작위성의 균형

단점:

- 고숙련 플레이어가 패턴 악용 가능
- 확률적 게임(포커 등)에는 부적합

3.4 적응형 난이도 조절 (Dynamic Difficulty Adjustment, DDA)

3.4.1 러버밴딩 (Rubber Banding)

개념: AI 속도를 플레이어와의 거리 차이에 따라 조절

사용 게임: Mario Kart 시리즈

수학적 모델:

$$AI\ Speed = BaseSpeed \times (1 + \alpha \times (Distance\ Diff / Max\ Distance))$$

여기서:

- α : 러버밴딩 강도 (0.1 ~ 0.5)
- 거리가 양수: AI 가 뒤처짐 → 속도 증가
- 거리가 음수: AI 가 앞섬 → 속도 감소

예제 3.9: 러버밴딩 파라미터

플레이어 거리	AI 속도 배수 ($\alpha=0.3$)	효과
+500m (뒤처짐)	1.15×	15% 빠름
+200m	1.06×	6% 빠름
0m (동률)	1.00×	기본 속도
-200m (앞섬)	0.94×	6% 느림
-500m	0.85×	15% 느림

장점: 항상 긴장감 유지, 역전 가능

단점: 과도하면 플레이어 노력이 무의미하게 느껴짐

3.4.2 플레이어 스킬 추정 (Player Skill Estimation)

측정 지표:

지표	계산 방법	응용
승률	(승리) / (총 게임)	전반적 난이도
정확도	(적중) / (발사)	적 회피 패턴
반응 시간	평균 입력 지연	적 공격 속도
대미지 비율	(가한 대미지) / (받은 대미지)	적 ATK/DEF 밸런스
생존 시간	평균 사망까지 시간	적 공격 빈도

스킬 지수 공식:

$$S = w1 \times \text{승률} + w2 \times \text{정확도} + w3 \times (1/\text{반응시간}) + w4 \times \text{대미지비율}$$

여기서 $w1 + w2 + w3 + w4 = 1$ (가중 평균)

난이도 조절:

$$\text{적 HP} = \text{기본 HP} \times (1 + \beta \times (S - 0.5))$$

$$\text{적 ATK} = \text{기본 ATK} \times (1 + \beta \times (S - 0.5))$$

여기서:

- β : 조절 강도 (0.2 ~ 0.8)
- S: 스킬 지수 (0.0 ~ 1.0)
- $S = 0.5$: 평균 플레이어
- $S > 0.5$: 숙련자 (난이도 증가)
- $S < 0.5$: 초보자 (난이도 감소)

예제 3.10: 스킬 기반 난이도

플레이어 스킬 $S = 0.70$ (숙련자), $\beta = 0.4$

- 적 HP 배수: $1 + 0.4(0.70 - 0.5) = 1.08$ (8% 증가)
- 적 ATK 배수: 1.08 (8% 증가)

3.4.3 베이지안 난이도 조절

베이즈 정리를 사용하여 플레이어 실력을 확률적으로 추정

모델 가설:

- H_1 : 플레이어는 초보자
- H_2 : 플레이어는 중급자
- H_3 : 플레이어는 고수

사전 확률: $P(H_1) = 0.30$, $P(H_2) = 0.50$, $P(H_3) = 0.20$

증거: 플레이어가 3 연속 승리 (E)

우도:

- $P(3 \text{ 연승} \mid \text{초보자}) = 0.10$
- $P(3 \text{ 연승} \mid \text{중급자}) = 0.40$
- $P(3 \text{ 연승} \mid \text{고수}) = 0.80$

베이즈 정리 적용:

가설	$P(E H)$	Prior	Product	Posterior
초보자	0.10	0.30	0.030	0.05
중급자	0.40	0.50	0.200	0.34
고수	0.80	0.20	0.160	0.61

정규화: $0.030 + 0.200 + 0.160 = 0.295$

결과: 고수일 사전 확률 20% → 3 연승 후 사후 확률 61%

→ AI 는 난이도를 크게 증가

3.4.4 몰입 이론 (Flow Theory)과 DDA

Csikszentmihalyi 의 Flow Channel:

난이도가 실력보다 너무 높으면: 불안 (Anxiety)

난이도가 실력보다 너무 낮으면: 지루함 (Boredom)

적절한 균형: 몰입 (Flow)

DDA 목표: 플레이어를 몰입 채널에 유지

수학적 표현:

$$Difficulty = Skill + \varepsilon$$

여기서 ε 은 작은 양수 (약간의 도전)

3.5 수학적 증명

정리 3.3: 베이즈 정리의 연쇄 법칙

$$P(A_1, A_2, \dots, A_n) = P(A_1) \times \prod_{i=2 \rightarrow n} P(A_i | A_1, \dots, A_{i-1})$$

증명 (귀납법):

기저 사례 (n=2): $P(A_1, A_2) = P(A_2 | A_1)P(A_1)$ ✓

귀납 가정: n=k 일 때 성립

귀납 단계: n=k+1 일 때

$$\begin{aligned} P(A_1, \dots, A_{k+1}) &= P(A_{k+1} | A_1, \dots, A_k) P(A_1, \dots, A_k) \\ &= P(A_1) \prod P(A_i | \dots) \quad \checkmark \end{aligned}$$

3.6 의사코드

알고리즘 3.2: 가중치 랜덤 선택 (이진 탐색)

```
WEIGHTED_RANDOM_SELECTION(items, weights, n)
    cumulative[0] ← weights[0]
    for i ← 1 to n-1:
        cumulative[i] ← cumulative[i-1] + weights[i]

    totalWeight ← cumulative[n-1]
    randomValue ← RANDOM(0, totalWeight)

    left ← 0
    right ← n-1
    while left < right:
        mid ← [(left + right) / 2]
        if randomValue < cumulative[mid]:
            right ← mid
        else:
            left ← mid + 1

    return items[left]
```

알고리즘 3.3: PRD 시스템

```
PRD_INITIALIZE(targetProbability)
    initialProb ← 0.019 × targetProbability
    increment ← initialProb
    currentProb ← initialProb
    return (initialProb, increment, currentProb)

PRD_TRY_PROC(currentProb, initialProb, increment)
    randomValue ← RANDOM(0, 1)
    if randomValue < currentProb:
        currentProb ← initialProb
        return (TRUE, currentProb)
    else:
        currentProb ← currentProb + increment
        return (FALSE, currentProb)
```

알고리즘 3.4: 스킬 기반 난이도 조절

```
UPDATE_DIFFICULTY(playerStats,  $\beta$ )  
  winRate  $\leftarrow$  playerStats.wins / playerStats.totalGames  
  accuracy  $\leftarrow$  playerStats.hits / playerStats.shots  
  damageRatio  $\leftarrow$  playerStats.damageDealt / playerStats.damageTaken  
  
  skillIndex  $\leftarrow$   $0.4 \times \text{winRate} + 0.3 \times \text{accuracy} + 0.3 \times \text{damageRatio}$   
  difficultyMultiplier  $\leftarrow$   $1 + \beta \times (\text{skillIndex} - 0.5)$   
  
  enemy.health  $\leftarrow$  baseHealth  $\times$  difficultyMultiplier  
  enemy.attackPower  $\leftarrow$  baseAttack  $\times$  difficultyMultiplier  
  enemy.attackFrequency  $\leftarrow$  baseFrequency  $\times$  difficultyMultiplier  
  
  return difficultyMultiplier
```

3.7 요약

핵심 개념 정리

- 확률 공간: $(\Omega, \mathcal{F}, P)$ 구조화된 프레임워크
- 조건부 확률 $P(A|B)$: 전술 AI 의 핵심
- 베이즈 정리: 사전 $\rightarrow$ 사후 확률 업데이트
- 균등 분포: 공정, 정규 분포: 자연스러움
- 지수 분포: 무기역성 이벤트 모델링
- 기댓값과 분산: 밸런싱 핵심 지표
- 가중치 랜덤: $O(\log n)$ 이진 탐색
- PRD: 체계적인 불공정 방지
- 러버밴딩: 거리 기반 AI 속도 조절
- 플레이어 스킬 추정: 다중 지표 합성
- 베이지안 난이도: 확률적 실력 추적
- 몰입 이론: 난이도를 실력보다 약간 높게

사용 가이드라인

베이즈 정리 사용 시기: 불완전 정보, AI 추론 (RTS 정찰)

베이즈 정리 회피 시기: 완전 정보 게임, 단순 무작위

PRD 사용 시기: 플레이어 체감 확률 중요 (치명타, 드롭)

PRD 회피 시기: 완벽한 무작위 게임 (포커, 주사위), 고수 악용 위험

DDA 사용 시기: 싱글플레이, 다양한 실력 수용

DDA 회피 시기: 경쟁 멀티플레이 (공정성), 노력이 보상받아야 할 때

3.8 연습문제

A. 개념 확인

1. 카드 게임 확률 공간 정의 및 계산
2. 균등 분포와 정규 분포 비교 (각 3 가지 예시)
3. PRD 의 장단점 설명

B. 계산/증명

4. FPS 조건부 확률 계산
5. 아이템 드롭 기댓값과 분산
6. 이항 분포: 75% 명중률 $\times$ 10 회 공격
7. RTS 베이스 정리 적 본진 위치
8. 가중치 랜덤 CDF 구성

C. 적용

9. PRD 15% 목표 파라미터 계산
10. DDA 스킬 기반 스케일링
11. 러버밴딩 분석
12. 복잡한 확률 독립성

D. 구현/디버깅

프로젝트: 확률 시스템 수학적 설계

요구사항:

- 1. 아이템 드롭 시스템 (30 점): ≥ 5 아이템 등급, 가중치 및 확률, 기댓값/분산, 100 드롭 평균값
- 2. PRD 시스템 (30 점): 25% 목표 PRD 파라미터, 1000 회 시뮬레이션, 분포 비교, 최대 연속 실패
- 3. 적응형 난이도 (30 점): 플레이어 스킬 공식, 난이도 조절 함수, 파라미터 테이블
- 4. 보고서 (10 점): 수학적 모델, 시뮬레이션 결과, 설계 정당화, 개선 방향

Chapter 4. 센서와 지각 시스템

4.0 도입과 동기

왜 센서가 중요한가?

문제: 센서 없는 AI 는 멍청해 보인다

예시: Splinter Cell - AI 가 벽 너머를 보는 경우

플레이어 몰입감 저하: "AI 가 치트를 쓴다"

AAA 게임 사례:

Splinter Cell: FOV 와 레이캐스팅 공정성

- 경비병은 시야각 내에서만 감지
- 장애물이 시선을 차단
- 플레이어: "공정한 게임"이라고 느낌

The Last of Us: 청각 + 메모리 시스템

- 발소리가 근처 AI 경보 트리거
- AI 는 소리 발생 위치로 이동 (플레이어 현재 위치 아님)
- 플레이어가 소리 후 이동 시 AI 는 빈 위치 조사
- 자연스러운 행동 패턴

FEAR: 센서 융합 + 통신

- 한 병사가 플레이어 발견 → 무전으로 다른 병사에게 알림
- 무전 받은 AI: 정확도 낮음 (간접 정보)
- 직접 관찰 AI: 정확한 조준
- 자연스러운 행동 다양성

본 챕터 내용:

- 1. 시각 센서: FOV + 레이캐스팅
- 2. 청각 센서: 거리 감쇠
- 3. 센서 융합: 가중치 신뢰도
- 4. 메모리 시스템: Last Known Position (LKP)
- 5. 성능 최적화: 폴링 vs 이벤트

4.1 센서 모델링 이론

4.1.1 센서의 정의와 분류

정의 4.1: 센서

게임 세계의 상태를 관찰하여 에이전트에게 정보를 제공하는 소프트웨어 컴포넌트

관찰 범위별 분류:

분류	센서 타입	설명	게임 예시
범위	지역	근처 환경만	FOV, 청각 범위
	전역	전체 맵 정보	미니맵, 전략 AI
데이터 타입	이산	불린 판단	플레이어 감지 여부
	연속	실수값 측정	거리, 각도, 볼륨
신뢰도	완벽	100% 정확도	체스 AI (완전 정보)
	불완전	노이즈/불완전	전장의 안개, 주변 소음

4.1.2 센서 모델의 수학적 표현

센서 모델: 실제 상태 s 와 관찰 o 의 관계

$$P(o|s) = \text{센서 관찰 확률}$$

완벽한 센서: $P(o|s) = 1$ (상태 s 가 참이면 항상 o 관찰)

불완전한 센서: $P(o|s) < 1$ (노이즈, 가림, 거리 감쇠, 조명)

예제 4.1: 시각 센서 모델

$$P(\text{타겟 보임} | \text{타겟 존재}) = P_{FOV} \times P_{\text{거리}} \times P_{\text{가림}} \times P_{\text{조명}}$$

4.1.3 센서 노이즈와 불확실성

노이즈 모델:

1. 가우시안 노이즈: 측정 오차가 정규 분포

$$o = s + N(0, \sigma^2)$$

예: 소리 방향 추정 $\pm 10^\circ$ 오차

2. 거리 감쇠: 거리에 따라 정확도 감소

$$P(\text{감지}) = 1.0 \quad (d \leq r_{\min})$$

$$P(\text{감지}) = 1.0 - (d - r_{\min}) / (r_{\max} - r_{\min}) \quad (r_{\min} < d \leq r_{\max})$$

$$P(\text{감지}) = 0.0 \quad (d > r_{\max})$$

3. False Positive/Negative:

- False Positive: 없는데 있다고 감지 (오탐)
- False Negative: 있는데 못 감지 (미탐)

4.1.4 폴링(Polling) vs 이벤트(Event) 방식

폴링 방법:

```
void Update()
{
    foreach (GameObject enemy in allEnemies)
    {
        if (IsInSightRange(enemy))
        {
            // 처리
        }
    }
}
```

복잡도: $O(n \times m)$ (n =AI 수, m =타겟 수)

장점: 단순, 예측 가능

단점: 대규모 확장 시 높은 비용

이벤트 방법:

```
public void OnSoundEvent(Vector3 soundPosition, float volume)
{
    float distance = Vector3.Distance(transform.position, soundPosition);
    if (volume / distance > hearingThreshold)
    {
        // 처리
    }
}
```

복잡도: $O(k)$ (k =이벤트 수)

장점: 데이터 존재 시만 필요, $O(k)$ 이벤트

단점: 이벤트 시스템 인프라 필요, 디버깅 복잡

성능 비교:

시나리오	Polling	Event
50 AI, 100 타겟, 60 FPS	300,000 연산/초	~100 이벤트/초
CPU 부하	매우 높음	낮음
지연 시간	1 프레임 (~16ms)	즉시

현대 엔진 (Unity, Unreal): 이벤트 기반 선호 (최적화)

4.2 시각 시스템

4.2.1 시야각 (Field of View, FOV) 모델

파라미터:

파라미터	변수명	일반 값	설명
시야 거리	viewRadius	15-30m	최대 감지 거리
시야각	viewAngle	90-120°	전방 원뿔 폭
눈 위치	eyePosition	pos + (0, 1.6, 0)	레이 시작점

수학적 모델: 타겟이 FOV 내에 있으려면:

1. 거리 체크:

$$d = \|p_{target} - p_{AI}\| \leq r_{view}$$

2. 각도 체크 (내적):

$$\cos(\theta) = (f \cdot d) / (\|f\| \cdot \|d\|) \geq \cos(\alpha/2)$$

여기서:

- f: AI 전방 벡터
- d: 타겟 방향 벡터
- α: FOV 각도 (예: 120°)

내적 해석:

- f · d = 1.0: 정면 (0°)
- f · d = 0.0: 직각 (90°)
- f · d = -1.0: 후방 (180°)

예: FOV 120° → 반각 60° → cos(60°) = 0.5 임계값

의사코드:

```
IS_IN_FOV(AI, target, viewRadius, viewAngle)
    dirToTarget ← target.position - AI.position
    distance ← LENGTH(dirToTarget)

    // 거리 체크
    if distance > viewRadius:
        return FALSE

    // 각도 체크
    dirToTarget ← NORMALIZE(dirToTarget)
    forwardDir ← AI.forward
    dotProduct ← DOT(forwardDir, dirToTarget)
    angleThreshold ← COS(viewAngle / 2)

    if dotProduct >= angleThreshold:
        return TRUE
    else:
        return FALSE
```

Unity C# 구현:

```
public bool IsInFOV(Transform target)
{
    Vector3 dirToTarget = (target.position - transform.position).normalized;
    float distance = Vector3.Distance(transform.position, target.position);

    // 거리 체크
    if (distance > viewRadius)
        return false;

    // 각도 체크
    float dot = Vector3.Dot(transform.forward, dirToTarget);
    float angleThreshold = Mathf.Cos(viewAngle * 0.5f * Mathf.Deg2Rad);
    return dot >= angleThreshold;
}
```

4.2.2 레이캐스팅 (Raycasting) 가시성 검증

문제: FOV 내에 있어도 장애물이 시선을 차단할 수 있음

해결책: 눈에서 타겟까지 레이를 쏘서 충돌 확인

레이 모델:

$$r(t) = Origin + t \times Direction, t \in [0, distance]$$

- Origin: AI 눈 위치
- Direction: 타겟 방향 정규화 벡터

의사코드:

```
HAS_LINE_OF_SIGHT(AI, target)
    eyePos ← AI.position + (0, eyeHeight, 0)
    dirToTarget ← target.position - eyePos
    distance ← LENGTH(dirToTarget)

    // 레이캐스트
    hit ← RAYCAST(eyePos, dirToTarget, distance)

    if hit.object == target:
        return TRUE // 타겟에 직접 명중
    else:
        return FALSE // 장애물 차단
```

최적화: Layer Mask

```
LayerMask obstacleMask = LayerMask.GetMask("Obstacles", "Walls");

if (Physics.Raycast(eyePosition, dirToTarget, out hit, distance,
    obstacleMask))
{
```

```
        return false; // 장애물 충돌, 타겟 아님
    }
    return true; // 시야 확보
```

지정된 레이어만 체크하여 불필요한 검사 회피

4.2.3 시각 시스템 통합

완전한 3 단계 감지:

```
DETECT_TARGET_VISUAL(AI, target)
// 1 단계: 거리 + 각도
if NOT IS_IN_FOV(AI, target):
    return FALSE

// 2 단계: 가림
if NOT HAS_LINE_OF_SIGHT(AI, target):
    return FALSE

// 3 단계: 메모리 기록
AI.perceivedTargets.ADD(target)
AI.lastSeenPosition[target] ← target.position
AI.lastSeenTime[target] ← CURRENT_TIME()

return TRUE
```

성능 최적화:

- 1. 조기 종료: 가장 저렴한 검사 먼저
- 2. 업데이트 간격 조절: 먼 타겟은 덜 자주 업데이트
- 3. 공간 분할: Octree/Quadtree 로 근처만 체크
- 4. LOD: 화면 밖 AI 는 간소화된 센서

업데이트 간격 예시:

```
float updateInterval;
float distanceToPlayer = Vector3.Distance(..., player.position);

if (distanceToPlayer < 10f)
    updateInterval = 0.0f;      // 매 프레임
else if (distanceToPlayer < 30f)
    updateInterval = 0.2f;      // 0.2 초마다
else
    updateInterval = 0.5f;      // 0.5 초마다
```

4.3 청각 시스템

4.3.1 음원 위치 추정

파라미터:

파라미터	설명	일반 값
Volume	음원 강도	0.0 ~ 1.0
Hearing Range	최대 감지 거리	20-50m
Threshold	최소 반응 볼륨	0.1 ~ 0.3

시각과 다르게: 청각은 전방향이지만 감쇠됨

4.3.2 거리 감쇠 모델

선형 감쇠:

$$V_{perceived} = V_{source} \times (1 - d/d_{max})$$

여기서:

- V_{source} : 초기 볼륨
- d : 소리까지 거리
- d_{max} : 청각 범위

예제 4.3:

거리	Volume ($V_{source}=1.0$, $d_{max}=50m$)
0m	1.0 (100%)
25m	0.5 (50%)
40m	0.2 (20%)
50m	0.0 (들리지 않음)

역제곱 감쇠 (물리적):

$$V_{perceived} = V_{source} / (1 + k \times d^2)$$

k = 감쇠 계수 (0.01 ~ 0.1)

더 현실적이지만 제어 어려움 → 게임에서는 선형 선호

의사코드:

```
PERCEIVE_SOUND(AI, soundEvent)
    distance ← DISTANCE(AI.position, soundEvent.position)

    // 거리 감쇠
    perceivedVolume ← soundEvent.volume × (1 - distance / AI.hearingRange)

    if perceivedVolume > AI.hearingThreshold:
        AI.lastHeardPosition ← soundEvent.position
        AI.lastHeardTime ← CURRENT_TIME()
        AI.alertLevel ← AI.alertLevel + perceivedVolume
        return TRUE
    else:
        return FALSE
```

4.3.3 장애물에 의한 소리 차단

감쇠 인자:

```
if RAYCAST(AI.position, soundPosition) hits obstacle:  
    perceivedVolume ← perceivedVolume × obstacleAttenuation  
end if
```

```
// obstacleAttenuation: 0.3 ~ 0.7 (재질별 다름)
```

예제 4.4: 복합 감쇠

- 초기: 1.0
- 거리 30m / 50m max: $1.0 \times (1 - 30/50) = 0.4$
- 벽 (감쇠 0.5): $0.4 \times 0.5 = 0.2$
- 최종 체감 볼륨: 0.2
- 임계값이 0.15 라면: 감지됨

4.3.4 추정 위치와 메모리

핵심 설계 원칙:

- AI 는 "플레이어 현재 위치"를 배우지 않음
- AI 는 "소리 발생 위치"를 배움
- 플레이어는 소리 낸 후 이동했을 수 있음

메모리 구조:

```
public class AIMemory
{
    public Vector3 lastHeardPosition; // 소리 발생원 (정확)
    public float lastHeardTime;      // 들은 시간
    public float confidence;          // 신뢰도 (시간 경과 시 감소)
}
```

스마트 AI 행동:

- 1. 위치 X 에서 발소리 듣기
- 2. 위치 X 로 이동
- 3. 도착 → 주변 탐색
- 4. 타겟 미발견 → 경계 유지 또는 순찰 복귀

나쁜 AI (회피해야 함):

- 1. 소리 듣기
- 2. 즉시 "플레이어 현재 위치" 앎
- 3. 벽 뒤 플레이어 위치로 직행
- 4. 플레이어는 치트라고 느낌

4.4 센서 융합과 메모리

4.4.1 정보의 통합 (Sensor Fusion)

신뢰도 기반 가중치:

센서	가중치 (w)	이유
시각	0.9	직접 관찰, 가장 정확
청각	0.6	방향/거리 모호성
통신	0.7	간접, 시간 지연
과거 메모리	0.3	시간 경과로 퇴화

융합 공식:

$$P(\text{타겟 존재}) = \Sigma(w_i \times P_i) / \Sigma w_i$$

예제 4.5: 센서 융합

센서	감지	확률	가중치	기여도
시각	예 (봄)	1.0	0.9	0.9
청각	예 (들음)	0.7	0.6	0.42
통신	예 (보고받음)	0.5	0.7	0.35
합계			2.2	1.67

$$P(\text{타겟}) = 1.67 / 2.2 \approx 0.76 = 76\%$$

해석: 76% 신뢰도로 타겟 존재

4.4.2 우선순위 규칙 (Priority Rules)

계층적 우선순위:

- 1. 시각 > 청각: 직접 관찰이 간접보다 우선
- 2. 최신 정보 > 오래된 메모리: 최근성 높은 가중치
- 3. 가까운 정보 > 먼 정보: 근접 타겟 우선

선택 의사코드:

```
SELECT_PRIORITY_TARGET(AI)
  targets ← COMBINE(visualTargets, auditoryTargets, ...)

  // 우선순위로 정렬
  SORT(targets, key = λt: (
    t.sensorType == VISION ? 100 : 0,      // 시각 우선
    1.0 / t.distance,                      // 가까운 것 우선
    1.0 / (CURRENT_TIME - t.lastSeen)     // 최신 우선
  ))

  return targets[0] // 최고 우선순위
```

4.4.3 단기 기억 (Short-Term Memory)

객체 연속성 문제:

- 타겟이 AI 시야를 벗어남
- AI 가 즉시 잊어서는 안 됨
- Last Known Position (LKP) 사용

LKP 구현:

```
public class TargetMemory
{
    public Transform target;
    public Vector3 lastKnownPosition;    // 마지막 관찰 위치
    public float lastSeenTime;           // 마지막 본 시간
    public float confidence;              // 신뢰도 (감소)

    public void Update()
    {
        float timeSinceLastSeen = Time.time - lastSeenTime;

        // 지수 감쇠
        confidence = Mathf.Exp(-timeSinceLastSeen / decayRate);

        // 10 초 후 완전 망각
        if (timeSinceLastSeen > 10f)
            confidence = 0;
    }
}
```

AI 행동 패턴:

- 1. 직접 시야: 타겟 능동 추적
- 2. 시야 상실: LKP 로 이동
- 3. LKP 도착: 주변 탐색
- 4. 일정 시간 미발견: 경계 유지 또는 순찰 복귀
- 5. 장시간 미발견: 완전 망각

LKP 주변 탐색 패턴:

```
SEARCH_AROUND_LKP(AI, lastKnownPosition)
    searchRadius ← 10.0
    searchPoints ← GENERATE_SPIRAL_POINTS(LKP, searchRadius, 8 개)

    for point in searchPoints:
        MOVE_TO(point)
        LOOK_AROUND() // 360° 회전 탐색
        if TARGET_FOUND():
            return

    AI.state ← PATROL // 미발견 시 순찰 복귀
```

신뢰도 감쇠 모델:

$$C(t) = C_0 \times e^{(-t/\tau)}$$

τ = 감쇠 상수 (예: 5 초)

4.4.4 장기 기억과 학습

위험 지도 학습:

```
Dictionary<Vector3Int, float> dangerMap; // 3D 그리드

void OnCombatEncounter(Vector3 position)
{
    Vector3Int gridPos = WorldToGrid(position);
    dangerMap[gridPos] = Mathf.Min(dangerMap[gridPos] + 0.1f, 1.0f);
}

float GetDangerLevel(Vector3 position)
{
    Vector3Int gridPos = WorldToGrid(position);
    return dangerMap.GetValueOrDefault(gridPos, 0.0f);
}
```

예제 4.6: 위치별 위험도

위치	과거 조우	위험 수준
좁은 복도	15 회	0.75 (높음)
넓은 광장	5 회	0.25 (낮음)
엄폐물 많은 구역	20 회	0.90 (매우 높음)

응용: AI 가 고위험 지역 진입 시 경계 증가

4.5 수학적 모델

정리 4.1: FOV 기하학적 조건

점 P가 에이전트 A의 FOV 내에 있을 필요충분조건:

$$\begin{aligned} \|AP\| &\leq r_{view} \\ (\underline{f}_A \cdot AP) / (\|\underline{f}_A\| \cdot \|AP\|) &\geq \cos(\alpha/2) \end{aligned}$$

증명:

- 1. 첫 번째 조건: 거리 제약 (단순)
- 2. 두 번째 조건: 내적 정의로부터
 - - 내적: $u \cdot v = \|u\| \|v\| \cos(\theta)$
 - - 정규화 벡터: $\hat{u} \cdot \hat{v} = \cos(\theta)$
 - - FOV 내 각도: $\theta \leq \alpha/2$
 - - 코사인 감소 $[0, \pi]$: $\cos(\theta) \geq \cos(\alpha/2)$
 - - 따라서: 내적 부등식 성립 $\square$

4.6 의사코드

알고리즘 4.1: 통합 시각 센서

```
VISUAL_SENSOR_UPDATE(AI)
    perceivedTargets ← EMPTY_LIST
    potentialTargets ← GET_ALL_ENEMIES_IN_GAME()

    for target in potentialTargets:
        // 1 단계: FOV 체크
        if NOT IS_IN_FOV(AI, target, viewRadius, viewAngle):
            continue

        // 2 단계: 레이캐스팅
        eyePos ← AI.position + (0, eyeHeight, 0)
        dirToTarget ← NORMALIZE(target.position - eyePos)
        distance ← DISTANCE(AI.position, target.position)
        hit ← RAYCAST(eyePos, dirToTarget, distance, obstacleMask)

        if hit.object ≠ target:
            continue // 가려짐

        // 3 단계: 메모리
        memory ← AI.targetMemories[target]
        memory.lastKnownPosition ← target.position
        memory.lastSeenTime ← CURRENT_TIME()
        memory.confidence ← 1.0
        memory.sensorType ← VISION

    perceivedTargets.ADD(target)

return perceivedTargets
```

알고리즘 4.2: 소리 이벤트 처리

```
ON_SOUND_EVENT(AI, soundEvent)
    distance ← DISTANCE(AI.position, soundEvent.position)

    // 거리 감쇠
    perceivedVolume ← soundEvent.volume × (1 - distance / AI.hearingRange)

    // 장애물 감쇠
    if RAYCAST(AI.position, soundEvent.position) hits obstacle:
        perceivedVolume ← perceivedVolume × 0.5

    // 임계값 체크
    if perceivedVolume < AI.hearingThreshold:
        return FALSE

    // 메모리 업데이트
    memory ← AI.soundMemory
    memory.lastHeardPosition ← soundEvent.position
    memory.lastHeardTime ← CURRENT_TIME()
    memory.confidence ← perceivedVolume
    memory.sensorType ← HEARING

    // 상태 전이
    if AI.state == PATROL:
        AI.state ← INVESTIGATE
        AI.destination ← soundEvent.position

    return TRUE
```

알고리즘 4.3: 센서 융합

```
FUSE_SENSOR_DATA(AI)
  allTargets ← COMBINE(visualTargets, auditoryTargets, communicatedTargets)

  for target in allTargets:
    visualData ← AI.visualMemory[target]
    auditoryData ← AI.auditoryMemory[target]
    commData ← AI.communicationMemory[target]

    // 가중 평균
    totalWeight ← 0
    totalConfidence ← 0

    if visualData exists:
      totalConfidence += 0.9 × visualData.confidence
      totalWeight += 0.9

    if auditoryData exists:
      totalConfidence += 0.6 × auditoryData.confidence
      totalWeight += 0.6

    if commData exists:
      totalConfidence += 0.7 × commData.confidence
      totalWeight += 0.7

    fusedConfidence ← totalConfidence / totalWeight

    // 우선순위 위치 (시각 > 청각 > 통신)
    if visualData exists:
      position ← visualData.lastKnownPosition
    else if auditoryData exists:
      position ← auditoryData.lastHeardPosition
    else:
      position ← commData.reportedPosition

    AI.fusedMemory[target] ← (position, fusedConfidence)
```

4.7 요약

핵심 개념 정리

- 센서: 환경 정보 제공 소프트웨어 컴포넌트
- Polling: 매 프레임 체크 ($O(n \times m)$) vs Event: 트리거 체크 ($O(k)$)
- FOV: 거리 + 각도 (내적) 체크
- 레이캐스팅: 가시선 검증, 가림 차단
- 청각: 거리 감쇠 + 장애물 요소
- 융합: 다중 센서 가중 평균
- 메모리: LKP 를 시간 감쇠 신뢰도로 추적
- 최적화: 업데이트 빈도, 공간 분할, 레이어 마스크

사용 가이드라인

Polling: 적은 AI (~10), 단순 구현 필요

Events: 많은 AI (50+), 출시 빌드 우선

시각 최적화: 거리 기반 업데이트 간격, quadtree, LOD

회피: 센서 시스템 없는 완전 정보 게임

알고리즘 요약

- 시각: FOV 체크 → 레이캐스팅 → 메모리
- 청각: 거리 감쇠 → 장애물 감쇠 → 경보
- 융합: 가중 신뢰도 계산
- 메모리: 지수 시간 감쇠, 탐색 패턴

4.8 연습문제

A. 개념 확인

1. 범위/타입/신뢰도/업데이트별 센서 분류
2. 폴링 vs 이벤트 비교 (장단점)
3. Last Known Position (LKP) 정의 및 중요성

B. 계산/증명

4. FOV 판단: 3 개 위치 (120° FOV, 20m 범위)
5. 선형 모델로 소리 감쇠
6. 센서 융합 신뢰도 계산
7. 시간에 따른 메모리 신뢰도 감쇠

C. 적용

8. 스텔스 게임 센서 시스템 설계
9. 3D 공간 FOV 벡터 기하학 유도

D. 구현

시각 센서 알고리즘 검증

요구사항:

- 1. 기본 구현 (40 점): FOV 체크, 레이캐스팅 가림, 시각화(gizmos), ≥ 3 동시 타겟
- 2. 최적화 (30 점): 거리 기반 업데이트 간격, Layer Mask, 조기 종료, 프로파일링
- 3. 메모리 시스템 (20 점): LKP 기록, 시간 기반 신뢰도 감소, 타겟 상실 시 탐색
- 4. 보고서 (10 점): 알고리즘 설명, 전후 성능, 구현 도전과제, 개선 방향

Part II 종합 정리

핵심 개념 요약

Chapter 3: 확률

- 확률 공간, 조건부 확률, 베이즈 정리
- 분포: 균등, 이항, 정규, 지수
- 가중치 랜덤 선택 ($O(\log n)$ 이진 탐색)
- PRD 시스템 (감정적 공정성)
- DDA (난이도를 실력에 맞춤)

Chapter 4: 센서

- 센서 모델링 (폴링 vs 이벤트)
- 시각: FOV (내적), 레이캐스팅
- 청각: 거리 감쇠, 장애물
- 융합: 가중 센서 신뢰도
- 메모리: LKP + 지수 감쇠

수학적 핵심

- Bayes: $P(A|B) = P(B|A)P(A)/P(B)$
- FOV: $\cos(\theta) = (\text{forward} \cdot \text{dirToTarget}) / (||\text{forward}|| \times ||\text{dirToTarget}||)$
- Hearing: $V_{\text{heard}} = V_{\text{source}} \times (1 - d/R_{\text{max}}) \times \text{obstacleAttenuations}$
- Fusion: $C_{\text{fused}} = (\sum w_i \times C_i) / (\sum w_i)$
- Memory: $C(t) = C_0 \times e^{(-t/\tau)}$

Part III. 이동 알고리즘 (군집/조향)

게임 인공지능

NPC의 움직임과 판단, 학습을 만드는 13 가지 알고리즘 패턴

Part III 개요

Part I에서는 FSM을 통한 의사결정을, Part II에서는 확률과 센서를 통한 환경 인지를 다뤘습니다. 하지만 "NPC가 어떻게 자연스럽게 움직이는가?"라는 핵심 질문에 아직 답하지 못했습니다.

Part III에서는 게임 AI의 또 다른 핵심 요소인 이동(Movement)을 다룹니다. 새 떼가 복잡한 패턴으로 날고, 물고기 떼가 일사불란하게 방향을 바꾸며, 좀비 무리가 물결처럼 플레이어를 압박하는 모습은 모두 군집 알고리즘(Flocking Algorithm)의 결과입니다. 또한 미사일이 목표를 추적하고, 자동차가 장애물을 회피하며, 캐릭터가 부드럽게 목적지에 도착하는 것은 조향 행동(Steering Behaviors)의 산물입니다.

본 Part에서는 Craig Reynolds의 Boids 모델부터 시작하여 현대 게임의 AAA급 이동 시스템까지, 이론적 배경과 수학적 모델링, Unity C# 구현, 성능 최적화까지 전 과정을 다룹니다.

학습 목표

본 Part 를 학습한 후 여러분은 다음을 수행할 수 있어야 합니다:

1. Boids 알고리즘의 3 가지 핵심 규칙을 이해하고 Unity C#로 구현할 수 있다.
2. 군집 행동의 창발성(Emergence)을 수학적으로 분석하고 게임에 적용할 수 있다.
3. $O(N^2)$ 복잡도 문제를 공간 분할로 $O(N)$ 으로 최적화할 수 있다.
4. 운동학적(Kinematic) 이동과 동역학적(Dynamic) 이동의 차이를 설명할 수 있다.
5. Seek, Pursue, Arrive, Evade 등 8 가지 기본 조향 행동을 구현할 수 있다.
6. 복수의 조향 행동을 가중 합, 우선순위, 블렌딩 방식으로 결합할 수 있다.

Part III 구조

Chapter	제목	주요 내용	분량
5	군집 알고리즘 (Flocking)	Boids 모델, 창발적 행동, 공간 분할 최적화	~30p
6	조향 행동 (Steering Behaviors)	기본 조향, 예측 기반, 장애물 회피, 경로 추종	~35p

Chapter 5. 군집 알고리즘 (Flocking)

5.1 Boids 알고리즘 이론

5.1.1 Craig Reynolds 의 Boids 모델

역사적 배경

1986 년 SIGGRAPH 학회에서 Craig Reynolds 가 발표한 Boids(Bird-oid Objects 의 줄임말) 모델은 인공 생명(Artificial Life) 연구의 기념비적 성과입니다. 당시 수백 마리의 새나 물고기를 애니메이션으로 표현하는 것은 각 개체에 복잡한 경로를 일일이 지정해야 하는 거의 불가능한 작업이었습니다.

Reynolds 의 혁신적 아이디어는 "지역 규칙(Local Rules)만으로 전역 패턴(Global Pattern)을 만들 수 있다"는 것이었습니다. 각 개체(Boid)는 주변 이웃만 보고 간단한 3 가지 규칙을 따르는데, 그 결과 마치 하나의 생명체처럼 움직이는 자연스러운 떼 행동이 창발(Emerge)합니다.

정의 5.1: 창발적 행동 (Emergent Behavior)

개별 에이전트의 단순한 지역 규칙 상호작용으로부터 예측하지 못한 복잡한 전역 패턴이 자발적으로 나타나는 현상

창발적 행동의 핵심 특징:

- 1. 지역 규칙 (Local Rules): 각 개체는 주변 이웃만 관찰
- 2. 전역 패턴 (Global Pattern): 때 전체가 조율된 것처럼 행동
- 3. 자기 조직화 (Self-organization): 중앙 통제 없이 질서 형성
- 4. 강건성 (Robustness): 일부 개체 제거되어도 전체 패턴 유지

게임 적용 사례

게임	대상	효과
Batman: Arkham Knight	박쥐 떼	동굴에서 수백 마리 박쥐 날아오름
Half-Life 2	새 떼	플레이어 접근 시 비둘기 흩어짐
Assassin's Creed	군중	시장 광장 자연스러운 군중 이동
World War Z	좀비 무리	수백 마리가 벽 타고 올라감
Ace Combat	전투기 편대	4-8 대 전투기 대형 유지
SimCity	차량 군집	도로 위 교통 흐름

5.1.2 세 가지 핵심 규칙

규칙 1: 분리 (Separation)

정의: 너무 가까운 이웃과 충돌하지 않도록 멀어지기

목적: 개체 간 최소 거리 유지, 뭉쳐서 하나로 보이는 것 방지

수학적 모델:

$$F_{separation} = \Sigma (position - neighbor.position).normalized / distance$$

의사코드:

```
function SEPARATION(boid, neighbors, radius):  
    steeringForce = Vector3.zero  
  
    for each neighbor in neighbors:  
        distance = |boid.position - neighbor.position|  
  
        if distance < radius AND distance > 0:  
            diff = normalized(boid.position - neighbor.position)  
            diff = diff / distance # 거리 역수로 가중  
            steeringForce += diff  
  
    return steeringForce
```

Unity C# 구현:

```
Vector3 Separation(List<Boid> neighbors)  
{  
    Vector3 steer = Vector3.zero;  
    int count = 0;  
  
    foreach (Boid other in neighbors)  
    {  
        float distance = Vector3.Distance(transform.position,  
other.transform.position);
```

```
        if (distance > 0 && distance < separationRadius)
        {
            Vector3 diff = transform.position - other.transform.position;
            diff.Normalize();
            diff /= distance; // 가까울수록 강한 힘
            steer += diff;
            count++;
        }
    }

    if (count > 0)
        steer /= count;

    return steer;
}
```

규칙 2: 정렬 (Alignment)

정의: 이웃들의 평균 속도 방향으로 맞추기

목적: 떼 전체가 같은 방향으로 이동하는 흐름 형성

수학적 모델:

$$F_{alignment} = avgVelocity.normalized \times maxSpeed - currentVelocity$$

의사코드:

```
function ALIGNMENT(boid, neighbors):
    avgVelocity = Vector3.zero

    for each neighbor in neighbors:
        avgVelocity += neighbor.velocity

    if neighbors.count > 0:
        avgVelocity /= neighbors.count
        avgVelocity = normalized(avgVelocity) × maxSpeed
        steeringForce = avgVelocity - boid.velocity

    return steeringForce
```

Unity C# 구현:

```
Vector3 Alignment(List<Boid> neighbors)
{
    Vector3 avgVelocity = Vector3.zero;

    foreach (Boid other in neighbors)
    {
        avgVelocity += other.velocity;
    }

    if (neighbors.Count > 0)
    {
        avgVelocity /= neighbors.Count;
```

```
        avgVelocity.Normalize();
        avgVelocity *= maxSpeed;

        Vector3 steer = avgVelocity - velocity;
        return steer;
    }

    return Vector3.zero;
}
```

규칙 3: 응집 (Cohesion)

정의: 이웃들의 질량 중심(Center of Mass)을 향해 이동

목적: 떼가 흩어지지 않고 하나의 그룹 유지

수학적 모델:

$$centerOfMass = \Sigma(neighbor.position) / N$$

$$desiredVelocity = (centerOfMass - position).normalized \times maxSpeed$$

$$F_{cohesion} = desiredVelocity - currentVelocity$$

의사코드:

```
function COHESION(boid, neighbors):
    centerOfMass = Vector3.zero

    for each neighbor in neighbors:
        centerOfMass += neighbor.position

    if neighbors.count > 0:
        centerOfMass /= neighbors.count
        desiredVelocity = normalized(centerOfMass - boid.position) ×
maxSpeed
        steeringForce = desiredVelocity - boid.velocity

    return steeringForce
```

Unity C# 구현:

```
Vector3 Cohesion(List<Boid> neighbors)
{
    Vector3 centerOfMass = Vector3.zero;

    foreach (Boid other in neighbors)
    {
        centerOfMass += other.transform.position;
    }
}
```

```
        if (neighbors.Count > 0)
        {
            centerOfMass /= neighbors.Count;
            Vector3 desired = (centerOfMass - transform.position).normalized *
maxSpeed;
            Vector3 steer = desired - velocity;
            return steer;
        }

        return Vector3.zero;
    }
```

표 5.1: 세 가지 규칙의 상호작용

규칙	효과	규칙이 없다면?
Separation	적절한 간격 유지	모든 개체가 한 점에 뭉침
Alignment	일관된 흐름 방향	무작위로 사방팔방 이동
Cohesion	그룹 응집력	개체들이 사방으로 흩어짐

사고 실험 5.1: 규칙 제거 효과

- Separation 만 제거: 모든 Boid 가 질량 중심으로 모여 겹침 → 하나의 점처럼 보임
- Alignment 만 제거: 간격은 유지되지만 방향이 제각각 → 흩어진 채 제자리 회전
- Cohesion 만 제거: 방향은 같지만 떼가 점점 멀어짐 → 결국 시야에서 사라짐

5.2 군집의 수학적 모델링

5.2.1 힘의 합성과 가중치

정의 5.2: 가중 합 (Weighted Sum)

$$F_{total} = w_s \cdot F_{separation} + w_a \cdot F_{alignment} + w_c \cdot F_{cohesion}$$

변수 정의:

- F_{total} : 최종 조향력 (Steering Force)
- w_s, w_a, w_c : 각 규칙의 가중치 (일반적으로 0.5 ~ 2.0)
- $F_{separation}, F_{alignment}, F_{cohesion}$: 각 규칙으로부터 계산된 힘

가중치 조절 효과

시나리오	w_s	w_a	w_c
느슨한 떼 (새, 나비)	2.0	1.0	1.0
밀집 떼 (물고기, 쯔미)	1.0	1.0	2.0
정렬 중시 (전투기 편대)	1.0	2.0	1.0
분산 배치 (분대 전개)	1.5	1.5	0.5

Unity C# 통합 구현:

```
public class Boid : MonoBehaviour
{
    [Header("이동 파라미터")]
    public float maxSpeed = 5f;
    public float maxForce = 0.5f;

    [Header("행동 가중치")]
    [Range(0f, 3f)] public float separationWeight = 1.5f;
    [Range(0f, 3f)] public float alignmentWeight = 1.0f;
    [Range(0f, 3f)] public float cohesionWeight = 1.0f;

    [Header("감지 파라미터")]
    public float neighborRadius = 5f;
    public float separationRadius = 2f;

    private Vector3 velocity;
    private List<Boid> neighbors;

    void Update()
    {
        neighbors = FindNeighbors();

        // 세 가지 힘 계산
        Vector3 sep = Separation(neighbors) * separationWeight;
        Vector3 ali = Alignment(neighbors) * alignmentWeight;
        Vector3 coh = Cohesion(neighbors) * cohesionWeight;

        // 힘 합성
        Vector3 totalForce = sep + ali + coh;

        // 힘 제한
        if (totalForce.magnitude > maxForce)
        {
            totalForce = totalForce.normalized * maxForce;
        }

        // 속도 업데이트
        velocity += totalForce * Time.deltaTime;

        // 속도 제한
        if (velocity.magnitude > maxSpeed)
        {
            velocity = velocity.normalized * maxSpeed;
        }
    }
}
```

```

        velocity = velocity.normalized * maxSpeed;
    }

    // 위치 업데이트
    transform.position += velocity * Time.deltaTime;

    // 방향 회전
    if (velocity.magnitude > 0.1f)
    {
        transform.forward = velocity.normalized;
    }
}

List<Boid> FindNeighbors()
{
    List<Boid> result = new List<Boid>();
    Collider[] colliders = Physics.OverlapSphere(transform.position,
neighborRadius);

    foreach (Collider col in colliders)
    {
        Boid other = col.GetComponent<Boid>();
        if (other != null && other != this)
        {
            result.Add(other);
        }
    }

    return result;
}

// Separation, Alignment, Cohesion 메서드는 앞서 정의한 대로
}

```

예제 5.1: 가중치 실험

시나리오 1: 공포에 질린 군중

$w_s = 2.5$ (충돌 회피 최우선), $w_a = 0.5$ (방향 신경 안 씀), $w_c = 0.3$ (뿔뿔이 흩어짐)

효과: 폭발 현장에서 사람들이 패닉 상태로 사방으로 도망

시나리오 2: 군사 편대

$w_s = 1.0$, $w_a = 2.5$ (방향 일치 중시), $w_c = 1.5$ (대형 유지)

효과: 전투기들이 리더를 따라 일사불란하게 대형 유지

시나리오 3: 좀비 무리

$w_s = 0.8$ (어느 정도 겹쳐도 OK), $w_a = 1.0$, $w_c = 2.0$ (뭉쳐서 이동)

효과: 물결처럼 밀집된 채로 플레이어를 압박

5.2.2 이웃 판정 (Neighborhood Detection)

정의 5.3: 이웃 조건

Boid j 가 Boid i 의 이웃이 되려면 다음 두 조건을 만족해야 합니다:

조건 1: 거리 조건

$$distance(i, j) \leq neighborRadius$$

조건 2: 시야각 조건 (Field of View, FOV)

$$angle(i.forward, j.position - i.position) \leq FOV / 2$$

시야각 (Field of View)

실제 동물의 시야각:

- 새: $180^\circ \sim 270^\circ$ (거의 전방위)
- 물고기: 약 300° (뒤쪽 작은 사각지대)
- 인간: 약 180° (정면 집중)

수학적 계산:

```
toNeighbor = neighbor.position - boid.position  
angle = arccos(dot(boid.forward, toNeighbor.normalized))
```

```
if angle ≤ FOV / 2:  
    neighbor 는 시야 내  
else:  
    neighbor 는 시야 밖
```

Unity C# 구현:

```
List<Boid> FindNeighborsWithFOV(float fov = 270f)
{
    List<Boid> result = new List<Boid>();
    Collider[] colliders = Physics.OverlapSphere(transform.position,
neighborRadius);

    foreach (Collider col in colliders)
    {
        Boid other = col.GetComponent<Boid>();
        if (other != null && other != this)
        {
            Vector3 toOther = other.transform.position - transform.position;
            float angle = Vector3.Angle(transform.forward, toOther);

            if (angle <= fov / 2f)
            {
                result.Add(other);
            }
        }
    }

    return result;
}

// 디버깅용 Gizmo 시각화
void OnDrawGizmosSelected()
{
    Gizmos.color = Color.yellow;
    Gizmos.DrawWireSphere(transform.position, neighborRadius);

    // FOV 원뿔 그리기
    Vector3 leftBoundary = Quaternion.Euler(0, -fov/2, 0) *
transform.forward * neighborRadius;
    Vector3 rightBoundary = Quaternion.Euler(0, fov/2, 0) *
transform.forward * neighborRadius;

    Gizmos.color = Color.green;
    Gizmos.DrawLine(transform.position, transform.position + leftBoundary);
    Gizmos.DrawLine(transform.position, transform.position + rightBoundary);
}
```

5.2.3 경계 처리 (Boundary Handling)

방법 1: 벽 반사 (Wall Bounce)

경계에 닿으면 속도를 반대 방향으로 반전

```
if (position.x > boundaryMax.x || position.x < boundaryMin.x)
    velocity.x *= -1;

if (position.y > boundaryMax.y || position.y < boundaryMin.y)
    velocity.y *= -1;

if (position.z > boundaryMax.z || position.z < boundaryMin.z)
    velocity.z *= -1;

// 위치를 경계 안으로 클램핑
position = Vector3.Clamp(position, boundaryMin, boundaryMax);
```

방법 2: 부드러운 회전 (Soft Turn)

경계 근처에 도달하면 중심을 향한 힘을 점진적으로 증가

```
Vector3 BoundaryForce()
{
    Vector3 force = Vector3.zero;
    float turnMargin = 5f; // 경계로부터 몇 미터 전에 회전 시작

    if (position.x > boundaryMax.x - turnMargin)
        force.x -= (position.x - (boundaryMax.x - turnMargin));

    if (position.x < boundaryMin.x + turnMargin)
        force.x += ((boundaryMin.x + turnMargin) - position.x);

    // y, z 축도 동일하게 처리

    return force;
}
```

방법 3: 토로이드 공간 (Toroidal Wrap)

Pac-Man 스타일: 한쪽 끝에서 나가면 반대쪽 끝에서 등장

```
if (position.x > boundaryMax.x)
    position.x = boundaryMin.x;
else if (position.x < boundaryMin.x)
    position.x = boundaryMax.x;
```

```
// y, z 축도 동일하게 처리
```

5.3 최적화와 응용

5.3.1 성능 문제: $O(N^2)$ 복잡도

문제 정의

순진한(Naive) 알고리즘에서 각 Boid 가 모든 다른 Boid 와 비교하면:

성능 표:

Boid 수	연산 횟수	성능 (60 FPS)
10	100	무시 가능
100	10,000	약간 느림
1,000	1,000,000	심각한 프레임 드롭
10,000	100,000,000	게임 멈춤

원인: 모든 쌍(Pair) 비교 $\rightarrow N \times (N-1) / 2 \approx O(N^2)$

5.3.2 공간 분할 (Spatial Partitioning)

정의 5.4: 그리드 분할 (Grid Partitioning)

3D 공간을 균일한 크기의 셀(Cell)로 나누고, 각 Boid 를 해당 셀에 등록하여 근처 셀만 검색

핵심 아이디어:

- 1. 공간을 cellSize 크기의 격자로 분할
- 2. 각 Boid 를 자신이 위치한 셀에 등록
- 3. 이웃 검색 시 자신의 셀 + 인접 8 개(2D) 또는 26 개(3D) 셀만 탐색

셀 좌표 계산:

$$cellX = \text{floor}(\text{boid.position.x} / \text{cellSize})$$

$$cellY = \text{floor}(\text{boid.position.y} / \text{cellSize})$$

$$cellZ = \text{floor}(\text{boid.position.z} / \text{cellSize})$$

복잡도 분석:

- 등록: $O(N)$ - 각 Boid 를 한 번씩 셀에 등록
- 이웃 검색: $O(k)$ - k 는 셀 당 평균 Boid 수
- 전체: $O(N \times k) \approx O(N)$ (k 가 상수일 때)

Unity C# 구현 - SpatialGrid 클래스:

```
using System.Collections.Generic;
using UnityEngine;

public class SpatialGrid
{
    private Dictionary<Vector3Int, List<Boid>> grid;
    private float cellSize;

    public SpatialGrid(float cellSize)
    {
        this.cellSize = cellSize;
        this.grid = new Dictionary<Vector3Int, List<Boid>>();
    }

    // 위치를 셀 좌표로 변환
    private Vector3Int GetCellCoords(Vector3 position)
    {
        return new Vector3Int(
            Mathf.FloorToInt(position.x / cellSize),
            Mathf.FloorToInt(position.y / cellSize),
            Mathf.FloorToInt(position.z / cellSize)
        );
    }

    // Boid 를 그리드에 등록
    public void Register(Boid boid)
    {
        Vector3Int cell = GetCellCoords(boid.transform.position);

        if (!grid.ContainsKey(cell))
        {
            grid[cell] = new List<Boid>();
        }

        grid[cell].Add(boid);
    }

    // 프레임마다 그리드 재구성
    public void Clear()
    {
        grid.Clear();
    }

    // 주변 이웃 검색 (자신 + 인접 26 개 셀)
```

```

public List<Boid> GetNeighbors(Boid boid, float radius)
{
    List<Boid> neighbors = new List<Boid>();
    Vector3Int centerCell = GetCellCoords(boid.transform.position);

    // 3x3x3 = 27 개 셀 탐색 (자신 포함)
    for (int x = -1; x <= 1; x++)
    {
        for (int y = -1; y <= 1; y++)
        {
            for (int z = -1; z <= 1; z++)
            {
                Vector3Int checkCell = centerCell + new Vector3Int(x, y,
z);

                if (grid.ContainsKey(checkCell))
                {
                    foreach (Boid other in grid[checkCell])
                    {
                        if (other != boid)
                        {
                            float distance = Vector3.Distance(
                                boid.transform.position,
                                other.transform.position
                            );

                            if (distance <= radius)
                            {
                                neighbors.Add(other);
                            }
                        }
                    }
                }
            }
        }
    }

    return neighbors;
}
}

```

FlockManager 에서 사용:

```
public class FlockManager : MonoBehaviour
{
    public List<Boid> allBoids;
    private SpatialGrid spatialGrid;

    void Start()
    {
        // neighborRadius × 2 가 이상적인 cellSize
        spatialGrid = new SpatialGrid(cellSize: 10f);
    }

    void Update()
    {
        // 매 프레임 그리드 재구성
        spatialGrid.Clear();

        foreach (Boid boid in allBoids)
        {
            spatialGrid.Register(boid);
        }

        // 각 Boid 에게 이웃 할당
        foreach (Boid boid in allBoids)
        {
            boid.neighbors = spatialGrid.GetNeighbors(boid,
            boid.neighborRadius);
        }
    }
}
```

최적 cellSize 선택:

- 너무 작으면: 셀 개수 증가 → Dictionary 오버헤드
- 너무 크면: 셀 당 Boid 수 증가 → 검색 비용 증가
- 권장: $cellSize = neighborRadius \times 2$

5.3.3 추가 최적화 기법

1. 업데이트 빈도 감소 (Update Throttling)

```
private int frameCounter = 0;

void Update()
{
    frameCounter++;

    // 이웃 재계산은 5 프레임마다
    if (frameCounter % 5 == 0)
    {
        neighbors = FindNeighbors();
    }

    // 이동은 매 프레임
    Vector3 force = CalculateFlockingForce(neighbors);
    ApplyForce(force);
}
```

2. Level of Detail (LOD)

```
float distanceToCamera = Vector3.Distance(transform.position,
Camera.main.transform.position);

if (distanceToCamera > 50f)
{
    // 먼 거리: 간소화된 업데이트
    UpdateSimple();
}
else
{
    // 가까운 거리: 전체 로직
    UpdateFull();
}
```

3. 멀티스레딩 (Job System)

```
using Unity.Jobs;
using Unity.Collections;
using Unity.Burst;

[BurstCompile]
struct FlockingJob : IJobParallelFor
{
    [ReadOnly] public NativeArray<Vector3> positions;
    [ReadOnly] public NativeArray<Vector3> velocities;
    public NativeArray<Vector3> forces;

    public void Execute(int index)
    {
        // 각 Boid 의 힘 계산 (병렬 처리)
        Vector3 force = CalculateForce(index);
        forces[index] = force;
    }
}
```

성능 비교 표:

방법	1000 Boids 시간	비고
순진한 $O(N^2)$	~150ms	사용 불가
공간 분할	~8ms	필수
+ 업데이트 빈도 감소	~2ms	권장
Job System + Burst	~0.5ms	AAA 급

5.3.4 군집 알고리즘의 게임 응용

응용 1: 리더 추종 (Leader Following)

```
public class LeaderFollower : Boid
{
    public Transform leader; // 지정된 리더
    public float followDistance = 5f;

    Vector3 FollowLeader()
    {
        Vector3 targetPos = leader.position - leader.forward *
followDistance;
        Vector3 desired = (targetPos - transform.position).normalized *
maxSpeed;
        return desired - velocity;
    }

    void Update()
    {
        Vector3 followForce = FollowLeader() * 2.0f; // 리더 추종 우선
        Vector3 separationForce = Separation(neighbors) * 1.0f;

        ApplyForce(followForce + separationForce);
    }
}

// 게임 예시: 동료 AI, 편대 비행
```

응용 2: 경로 추종 군집 (Path Following Flock)

```
public class PathFlocking : Boid
{
    public Transform[] waypoints;
    private int currentWaypoint = 0;

    Vector3 FollowPath()
    {
        Vector3 target = waypoints[currentWaypoint].position;
        float distance = Vector3.Distance(transform.position, target);

        if (distance < 2f)
        {

```

```

        currentWaypoint = (currentWaypoint + 1) % waypoints.Length;
    }

    Vector3 desired = (target - transform.position).normalized *
maxSpeed;
    return desired - velocity;
}

void Update()
{
    // 경로 + 군집 행동
    Vector3 pathForce = FollowPath() * 1.5f;
    Vector3 flockForce = CalculateFlockingForce(neighbors);

    ApplyForce(pathForce + flockForce);
}
}

// 게임 예시: RTS 유닛 이동, 행군하는 군대

```

응용 3: 포식자 회피 (Predator Evasion)

```
public class PreyBoid : Boid
{
    public Transform predator;
    public float dangerRadius = 10f;

    Vector3 FleePredator()
    {
        float distance = Vector3.Distance(transform.position,
predator.position);

        if (distance < dangerRadius)
        {
            Vector3 desired = (transform.position -
predator.position).normalized * maxSpeed;
            float panicFactor = 1f - (distance / dangerRadius); //
가까울수록 강함
            return (desired - velocity) * panicFactor * 3f;
        }

        return Vector3.zero;
    }

    void Update()
    {
        Vector3 fleeForce = FleePredator();
        Vector3 flockForce = CalculateFlockingForce(neighbors) * 0.5f; //
군집 약화

        ApplyForce(fleeForce + flockForce);
    }
}

// 게임 예시: 사냥감 AI, 도망치는 군중
```

응용 4: 장애물 회피 군집 (Obstacle Avoidance Flock)

```
Vector3 AvoidObstacles()
{
    Vector3 avoidance = Vector3.zero;
    float detectionDistance = 5f;

    RaycastHit hit;
```

```

Vector3[] directions = new Vector3[]
{
    transform.forward,
    transform.forward + transform.right * 0.5f,
    transform.forward - transform.right * 0.5f
};

foreach (Vector3 dir in directions)
{
    if (Physics.Raycast(transform.position, dir, out hit,
detectionDistance))
    {
        Vector3 away = transform.position - hit.point;
        avoidance += away.normalized / hit.distance;
    }
}

return avoidance;
}

void Update()
{
    Vector3 obstacleForce = AvoidObstacles() * 2.0f; // 최우선
    Vector3 flockForce = CalculateFlockingForce(neighbors);

    ApplyForce(obstacleForce + flockForce);
}

```

5.4 연습문제

문제 5.1 [개념 이해]

- (a) Boids 알고리즘의 세 가지 핵심 규칙을 설명하시오.
- (b) Alignment 규칙을 제거하면 어떤 현상이 발생하는지 서술하시오.

문제 5.2 [수학적 계산]

Boid A의 위치가 (0, 0, 0)이고, 주변 이웃 3개의 위치가 다음과 같을 때:

- B: (2, 0, 0)
- C: (-1, 1, 0)
- D: (0, -1, 1)

- (a) Separation 힘 벡터를 계산하시오 (separationRadius = 3)
- (b) Cohesion 힘 벡터를 계산하시오 (maxSpeed = 5)

문제 5.3 [설계 문제]

다음 3가지 게임 시나리오에 적합한 가중치 조합을 설계하고 그 이유를 설명하시오:

- (a) 공포 영화의 박쥐 떼 (빠르고 산발적)
- (b) 수족관의 물고기 떼 (밀집되고 느림)
- (c) 전투 헬기 편대 (정렬 중시, 일정 간격)

문제 5.4 [복잡도 분석]

(a) 1000 개 Boid 가 $O(N^2)$ 알고리즘으로 매 프레임(60 FPS) 연산하면 초당 몇 번의 거리 비교가 발생하는가?

(b) 공간 분할을 사용하고 Boid 가 균등 분포되어 셀 당 평균 10 개씩 있다면, 초당 연산 횟수를 추정하십시오.

문제 5.5 [구현 문제]

다음 기능을 Unity C#로 구현하십시오:

(a) FOV 120°를 고려한 이웃 검색

(b) 경계 부근에서 중심을 향한 힘 생성

(c) 포식자로부터 도망치는 Evade 행동

Chapter 6. 조향 행동 (Steering Behaviors)

6.1 운동학 vs 동역학

6.1.1 운동학적 이동 (Kinematic Movement)

정의 6.1: 운동학적 이동

힘과 가속도 없이 위치와 방향을 직접 수정하는 이동 방식

의사코드:

```
function KINEMATIC_UPDATE(agent, target, speed, deltaTime):  
    direction = normalized(target.position - agent.position)  
    agent.position += direction × speed × deltaTime  
    agent.rotation = direction # 즉시 회전
```

Unity C# 구현:

```
void KinematicMove(Vector3 target, float speed)  
{  
    Vector3 direction = (target - transform.position).normalized;  
    transform.position += direction * speed * Time.deltaTime;  
  
    if (direction.magnitude > 0.01f)  
    {  
        transform.forward = direction; // 즉각 회전  
    }  
}
```

장점:

- 1. 구현 단순

- 2. 예측 가능
- 3. 계산 부담 낮음

단점:

- 1. 회전이 뚝뚝 끊김 (90° 즉시 회전)
- 2. 정지/출발이 부자연스러움 (관성 없음)
- 3. 충돌 시 겹침 발생 가능

사용 사례:

2D 슈팅 게임, 탑다운 RPG, 단순 퍼즐 게임

비교 표:

특성	Kinematic	Dynamic (Steering)
위치 업데이트	$\text{position} += \text{direction} \times \text{speed}$	$\text{position} += \text{velocity}$
속도 변화	즉시	점진적 (가속도)
방향 전환	90° 즉시	부드러운 곡선
정지	즉시	관성으로 미끄러짐
복잡도	낮음	중간-높음
현실감	낮음	높음

6.1.2 동역학적 이동 (Dynamic Movement)

정의 6.2: 동역학적 이동

뉴턴의 제 2 법칙($F = ma$)을 기반으로 힘을 가하여 가속도를 만들고, 속도와 위치를 업데이트하는 방식

뉴턴의 제 2 법칙:

$$F = ma$$

$$a = F/m$$

물리 업데이트 공식:

$$acceleration = force / mass$$

$$velocity = velocity + acceleration \times \Delta t$$

$$position = position + velocity \times \Delta t$$

Unity C# 구현:

```
public class DynamicAgent : MonoBehaviour
{
    public float mass = 1.0f;
    public float maxSpeed = 10f;
    private Vector3 velocity = Vector3.zero;

    public void ApplyForce(Vector3 force)
    {
        Vector3 acceleration = force / mass;
        velocity += acceleration * Time.deltaTime;

        // 속도 제한
        if (velocity.magnitude > maxSpeed)
        {
            velocity = velocity.normalized * maxSpeed;
        }
    }
}
```

```
        // 위치 업데이트
        transform.position += velocity * Time.deltaTime;

        // 부드러운 회전
        if (velocity.magnitude > 0.1f)
        {
            transform.forward = velocity.normalized;
        }
    }
}
```

장점:

- 1. 자연스러운 가속/감속
- 2. 관성 표현 가능
- 3. 물리적으로 타당
- 4. 여러 힘 합성 가능

단점:

- 1. 구현 복잡
- 2. 파라미터 튜닝 필요
- 3. 정확한 목표 도달 어려움

사용 사례:

레이싱 게임, 비행 시뮬레이션, 현대 3D 액션, 물리 기반 퍼즐

6.2 기본 조향 행동

조향력 개념

정의 6.3: 조향력 (Steering Force)

$$\text{Steering Force} = \text{desired velocity} - \text{current velocity}$$

점진적으로 방향과 속도를 조절하며, 프레임마다 누적되어 부드러운 곡선 경로 생성

6.2.1 Seek (추적)

정의: 목표를 향해 최대 속도로 가속

알고리즘 (3 단계):

- 1. 목표를 향한 방향 벡터 계산
- 2. 그 방향으로 maxSpeed 의 desired velocity 생성
- 3. 현재 속도와의 차이를 조향력으로 반환

수학적 모델:

$$v_{desired} = (target - position).normalized \times v_{max}$$

$$F_{seek} = v_{desired} - v_{current}$$

의사코드:

```
function SEEK(agent, target, maxSpeed):  
    desired = normalized(target.position - agent.position) × maxSpeed  
    steeringForce = desired - agent.velocity  
    return steeringForce
```

Unity C# 구현:

```
Vector3 Seek(Vector3 targetPosition)  
{  
    Vector3 desired = (targetPosition - transform.position).normalized *  
    maxSpeed;  
    Vector3 steer = desired - velocity;  
    return steer;  
}
```

시각적 설명:

위치 A (현재) → 위치 B (목표)

현재 속도: 북동쪽으로 이동 중

원하는 속도: 동쪽 (목표 방향)

조향력: 남동쪽 약간 (속도를 동쪽으로 수정)

문제점: Overshoot (오버슈트)

목표를 지나쳐서 원을 그리며 맴도는 현상

6.2.2 Flee (도주)

정의: Seek 의 반대 - 목표로부터 멀어지기

수학적 모델:

$$F_{flee} = -F_{seek}$$

Unity C# 구현:

```
Vector3 Flee(Vector3 dangerPosition)
{
    Vector3 desired = (transform.position - dangerPosition).normalized *
maxSpeed;
    Vector3 steer = desired - velocity;
    return steer;
}
```

향상된 응용 - 범위 제한 도주:

```
Vector3 FleeWithRange(Vector3 dangerPosition, float panicRadius)
{
    float distance = Vector3.Distance(transform.position, dangerPosition);

    if (distance < panicRadius)
    {
        Vector3 desired = (transform.position - dangerPosition).normalized *
maxSpeed;
        Vector3 steer = desired - velocity;

        // 거리 역수로 강도 조절
        float intensity = 1f - (distance / panicRadius);
        return steer * intensity;
    }

    return Vector3.zero; // 안전 거리면 힘 없음
}
```

게임 응용:

- NPC 가 플레이어 접근 시 후퇴
- 체력 낮은 유닛 철수
- 폭탄 회피

6.2.3 Arrive (도착)

정의: 목표에 가까워질수록 감속하여 정확히 멈추기

핵심 아이디어: 목표까지 거리에 따라 **desired speed** 조절

알고리즘 (4 단계):

- 1. 목표까지 거리 계산
- 2. $distance < slowingRadius$ 이면 속도를 비례적으로 줄임
- 3. $distance > slowingRadius$ 이면 $maxSpeed$ 사용
- 4. 현재 속도와의 차이로 조향력 계산

수학적 모델:

$$v_{desired_magnitude} = v_{max} \quad (d > r_{slow})$$

$$v_{desired_magnitude} = v_{max} \times (d / r_{slow}) \quad (d \leq r_{slow})$$

의사코드:

```
function ARRIVE(agent, target, slowingRadius, maxSpeed):
    toTarget = target.position - agent.position
    distance = length(toTarget)

    if distance < slowingRadius:
        desiredSpeed = maxSpeed * (distance / slowingRadius)
    else:
        desiredSpeed = maxSpeed

    desired = normalized(toTarget) * desiredSpeed
    steeringForce = desired - agent.velocity
    return steeringForce
```

Unity C# 구현:

```
Vector3 Arrive(Vector3 targetPosition, float slowingRadius)
{
    Vector3 toTarget = targetPosition - transform.position;
    float distance = toTarget.magnitude;

    if (distance < 0.1f) // 거의 도착
        return Vector3.zero;

    float targetSpeed;

    if (distance < slowingRadius)
    {
        // 감속 구간
        targetSpeed = maxSpeed * (distance / slowingRadius);
    }
    else
    {
        // 최대 속도
        targetSpeed = maxSpeed;
    }

    Vector3 desired = toTarget.normalized * targetSpeed;
    Vector3 steer = desired - velocity;

    return steer;
}
```

시각적 설명:

거리 진행: 10m → 5m → 2.5m → 1m → 0.5m → 0m

속도 변화: 100% → 100% → 50% → 20% → 10% → 0%

Seek vs Arrive 비교:

측면	Seek	Arrive
목표 접근 속도	항상 maxSpeed	거리 따라 감소

목표 도착	오버슈트	정확히 멈춤
궤적	거의 직선	곡선 후 직선
사용 사례	지속적 추격	정확한 위치 결정

게임 응용:

- NPC 가 특정 위치로 걸어가기
- 드론이 웨이포인트에 착륙
- 카메라 부드러운 이동

6.3 예측 기반 고급 조향

6.3.1 Pursue (예측 추적)

문제 정의

단순 Seek 는 현재 위치만 추적 → 빠르게 움직이는 목표에 비효율적

해결책: 목표의 미래 위치 예측

알고리즘 (4 단계):

- 1. 목표까지 거리 계산
- 2. 예측 시간 = 거리 / maxSpeed
- 3. 미래 위치 = 현재 위치 + 속도 × 예측 시간
- 4. 미래 위치를 향해 Seek

수학적 모델:

$$t_{predict} = |target_position - position| / v_{max}$$
$$P_{future} = target_position + target_velocity \times t_{predict}$$
$$F_{pursue} = Seek(P_{future})$$

의사코드:

```
function PURSUE(agent, target, maxSpeed):  
    distance = length(target.position - agent.position)  
    predictionTime = distance / maxSpeed  
  
    futurePosition = target.position + target.velocity × predictionTime  
    return SEEK(agent, futurePosition, maxSpeed)
```


Unity C# 구현:

```
Vector3 Pursue(Transform target)
{
    Vector3 toTarget = target.position - transform.position;
    float distance = toTarget.magnitude;

    // 예측 시간 계산
    float predictionTime = distance / maxSpeed;

    // 목표의 속도 가져오기
    Vector3 targetVelocity = GetTargetVelocity(target);

    // 미래 위치 예측
    Vector3 futurePosition = target.position + targetVelocity *
predictionTime;

    // 미래 위치를 향해 Seek
    return Seek(futurePosition);
}

// 목표 속도 추출 (Rigidbody 또는 프레임 간 차이)
Vector3 GetTargetVelocity(Transform target)
{
    Rigidbody rb = target.GetComponent<Rigidbody>();
    if (rb != null)
    {
        return rb.velocity;
    }
    else
    {
        // 프레임 간 위치 차이로 추정
        return (target.position - previousTargetPosition) / Time.deltaTime;
    }
}
```

최적화 - 예측 시간 제한:

```
float predictionTime = Mathf.Min(distance / maxSpeed, 2.0f); // 최대 2 초
```

너무 먼 미래를 예측하면 오히려 부정확

게임 응용:

- 미사일이 비행기 추적
- 축구 게임 패스 예측
- FPS AI 의 조준 선행

6.3.2 Evade (예측 회피)

정의: Pursue 의 반대 - 미래 적 위치를 예측하여 도망

수학적 모델:

$$F_{evade} = Flee(P_{future})$$

Unity C# 구현:

```
Vector3 Evade(Transform threat)
{
    Vector3 toThreat = threat.position - transform.position;
    float distance = toThreat.magnitude;

    float predictionTime = distance / maxSpeed;
    predictionTime = Mathf.Min(predictionTime, 1.5f); // 제한

    Vector3 threatVelocity = GetTargetVelocity(threat);
    Vector3 futurePosition = threat.position + threatVelocity *
predictionTime;

    // 미래 위치로부터 도망
    return Flee(futurePosition);
}
```

게임 응용:

- 유도 미사일 회피
- 스텔스 게임에서 플레이어 공격 회피
- NPC 가 포식자 탈출

6.3.3 장애물 회피 (Obstacle Avoidance)

문제

Seek/Pursue 는 장애물을 무시하여 충돌 발생

해결책: 수염(Whiskers) = 여러 방향 레이캐스트로 예측 감지

알고리즘 (4 단계):

- 1. 여러 전방 방향으로 레이 발사
- 2. 가장 가까운 장애물 충돌점 찾기
- 3. 충돌 반대 방향으로 회피력 생성
- 4. 거리 역수로 회피 강도 조절

레이 구성 전략:

- 중앙 레이: forward
- 좌측 레이: forward + left × 0.5
- 우측 레이: forward + right × 0.5
- 상단 레이 (3D): forward + up × 0.5

의사코드:

```
function AVOID_OBSTACLES(agent, detectionDistance):  
    avoidanceForce = Vector3.zero  
    closestDistance = INFINITY  
  
    directions = [forward, forward+left×0.5, forward+right×0.5]  
  
    for each direction in directions:  
        hit = RAYCAST(agent.position, direction, detectionDistance)
```

```
    if hit.exists AND hit.distance < closestDistance:
        closestDistance = hit.distance
        avoidanceForce = (agent.position - hit.point).normalized
        avoidanceForce *= (1.0 - hit.distance / detectionDistance)

return avoidanceForce
```

Unity C# 구현:

```
Vector3 AvoidObstacles(float detectionDistance, LayerMask obstacleLayer)
{
    Vector3 avoidance = Vector3.zero;
    float closestDistance = Mathf.Infinity;
    Vector3 closestPoint = Vector3.zero;

    // 여러 방향 레이
    Vector3[] directions = new Vector3[]
    {
        transform.forward,
        transform.forward + transform.right * 0.5f,
        transform.forward - transform.right * 0.5f,
        transform.forward + transform.up * 0.3f
    };

    foreach (Vector3 dir in directions)
    {
        RaycastHit hit;
        if (Physics.Raycast(transform.position, dir.normalized, out hit,
            detectionDistance, obstacleLayer))
        {
            if (hit.distance < closestDistance)
            {
                closestDistance = hit.distance;
                closestPoint = hit.point;
            }

            // 디버그 시각화
            Debug.DrawLine(transform.position, hit.point, Color.red);
        }
        else
        {
            Debug.DrawRay(transform.position, dir.normalized *
detectionDistance,
                        Color.green);
        }
    }

    // 가장 가까운 장애물에 대한 회피력
    if (closestDistance < Mathf.Infinity)
    {
        avoidance = (transform.position - closestPoint).normalized;

        // 가까울수록 강한 힘
    }
}
```

```
        float strength = 1f - (closestDistance / detectionDistance);  
        avoidance *= strength;  
    }  
  
    return avoidance;  
}
```

고급 기법 - 표면 법선(Surface Normal) 사용:

```
if (Physics.Raycast(transform.position, dir.normalized, out hit,
detectionDistance))
{
    // 충돌 표면의 법선 벡터 사용 (더 자연스러운 회피)
    avoidance = hit.normal;
}
```

성능 최적화:

- 1. Layer Masking: "Obstacle" 레이어만 체크
- 2. 업데이트 빈도 감소: 5 프레임마다 재계산
- 3. 레이 개수 조절: 먼 거리에서는 레이 감소

게임 응용:

- 군중 시뮬레이션 벽 회피
- 레이싱 게임 경로 유지
- 비행 시뮬레이션 장애물

6.4 경로 추종 (Path Following)

6.4.1 기본 웨이포인트 추종

배경: A* 경로탐색은 웨이포인트 시퀀스 생성

알고리즘 (3 단계):

- 1. 현재 웨이포인트를 향해 Arrive
- 2. 충분히 가까워지면 다음 웨이포인트로 전환
- 3. 모든 웨이포인트 완료 시 종료

Unity C# 구현 - PathFollower 클래스:

```
public class PathFollower : MonoBehaviour
{
    public Transform[] waypoints;
    public float waypointRadius = 2f;
    public float maxSpeed = 5f;
    public float slowingRadius = 5f;

    private int currentWaypoint = 0;
    private Vector3 velocity;

    void Update()
    {
        if (currentWaypoint >= waypoints.Length)
        {
            // 경로 완료
            return;
        }

        Vector3 target = waypoints[currentWaypoint].position;
        float distance = Vector3.Distance(transform.position, target);

        // 웨이포인트 도달 체크
        if (distance < waypointRadius)
        {
```

```

        currentWaypoint++;
        return;
    }

    // Arrive 로 이동
    Vector3 steer = Arrive(target, slowingRadius);
    ApplyForce(steer);
}

Vector3 Arrive(Vector3 target, float slowingRadius)
{
    // 앞서 구현한 Arrive 메서드
}
}

```

문제점: 각 웨이포인트에서 회전할 때 각진 경로

6.4.2 부드러운 경로 추종 (Smooth Path Following)

원리: 웨이포인트 사이의 보이지 않는 선을 따라가기

알고리즘 (4 단계):

- 1. 현재 웨이포인트와 다음 웨이포인트 사이 선분 생성
- 2. 에이전트에서 선분까지 최소 거리 계산
- 3. pathRadius 밖이면 경로로 되돌아가는 힘
- 4. 경로 방향 앞쪽을 미리 보고(look-ahead) 부드러운 회전

수학:

경로 방향: $(\text{nextWP} - \text{currentWP}).\text{normalized}$

에이전트 투영: $\text{dot}(\text{toAgent}, \text{pathDirection})$

선분 위 가장 가까운 점: $\text{currentWP} + \text{pathDirection} \times \text{projection}$

경로로부터 거리: $\text{distance}(\text{position}, \text{closestPoint})$

Unity C# 구현 - FollowPath 메서드:

```
Vector3 FollowPath(float pathRadius, float lookAheadDistance)
{
    Vector3 currentWP = waypoints[currentWaypoint].position;
    Vector3 nextWP = waypoints[currentWaypoint + 1].position;

    // 경로 방향
    Vector3 pathDirection = (nextWP - currentWP).normalized;

    // 에이전트를 경로에 투영
    Vector3 toAgent = transform.position - currentWP;
    float projection = Vector3.Dot(toAgent, pathDirection);
```

```

// 경로 위 가장 가까운 점
Vector3 closestPoint = currentWP + pathDirection * projection;

// 경로로부터 거리
float distanceFromPath = Vector3.Distance(transform.position,
closestPoint);

Vector3 steer = Vector3.zero;

// 경로 밖이면 되돌아가기
if (distanceFromPath > pathRadius)
{
    steer = Seek(closestPoint);
}

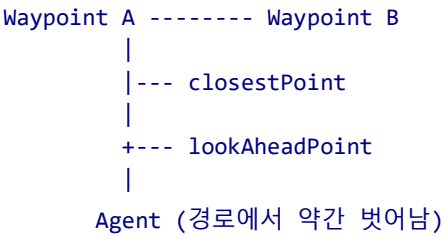
// Look-ahead: 경로 앞쪽을 향해 Seek
Vector3 lookAheadPoint = closestPoint + pathDirection *
lookAheadDistance;
steer += Seek(lookAheadPoint) * 0.5f;

// 다음 웨이포인트 전환 체크
if (projection > Vector3.Distance(currentWP, nextWP) - 1f)
{
    currentWaypoint++;
}

return steer;
}

```

시각적 다이어그램:



게임 응용:

- RTS 유닛이 A* 경로를 따라 이동
- 레이싱 게임 트랙 추종
- 경비병 순찰 경로

6.5 행동 결합 (Combining Behaviors)

문제: 복잡한 AI 는 여러 행동을 동시에 필요로 함

예: 장애물 회피 + 적 추격

6.5.1 방법 1: 가중 합 (Weighted Sum)

접근법: 각 힘에 가중치를 곱한 후 모두 더함

수학적 모델:

$$F_{total} = w1 \cdot F1 + w2 \cdot F2 + \dots + wn \cdot Fn$$

Unity C# 구현:

```
void Update()
{
    Vector3 seekForce = Seek(target.position) * 1.0f;
    Vector3 avoidForce = AvoidObstacles(5f, obstacleMask) * 2.0f;
    Vector3 separationForce = Separation(neighbors) * 1.5f;

    Vector3 totalForce = seekForce + avoidForce + separationForce;

    // 힘 크기 제한
    if (totalForce.magnitude > maxForce)
    {
        totalForce = totalForce.normalized * maxForce;
    }

    ApplyForce(totalForce);
}
```

장점:

- 1. 구현 간단

- 2. 부드러운 움직임

단점:

- 1. 힘이 상쇄될 수 있음 (Seek + Flee 동시 = 정지)
- 2. 가중치 튜닝 어려움

6.5.2 방법 2: 우선순위 기반 (Prioritized Dithering)

접근법: 우선순위 순으로 계산, 첫 번째 non-zero 힘 사용

알고리즘 (4 단계):

- 1. 행동을 우선순위 순으로 나열
- 2. 각 행동의 힘 계산
- 3. 힘 > 임계값이면 즉시 반환
- 4. 아니면 다음 우선순위 행동 시도

Unity C# 구현 - PrioritizedSteering:

```
Vector3 PrioritizedSteering()
{
    Vector3 force;

    // 우선순위 1: 장애물 회피 (가장 중요)
    force = AvoidObstacles(5f, obstacleMask);
    if (force.magnitude > 0.1f)
        return force;

    // 우선순위 2: 적 미사일 회피
    force = Evade(missile);
    if (force.magnitude > 0.5f)
        return force;

    // 우선순위 3: 동료와 분리
    force = Separation(neighbors);
    if (force.magnitude > 0.3f)
        return force;

    // 우선순위 4: 기본 행동 (Pursue)
    force = Pursue(enemy);
    return force;
}
```

우선순위 표 예시:

우선순위	행동	임계값	상황
1	Obstacle Avoidance	벽 2m 거리	충돌 직전
2	Evade	적 5m 거리	미사일 접근
3	Separation	동료 1m 거리	유닛 겹침
4	Pursue	항상	기본 행동

장점:

- 1. 명확한 행동 우선순위
- 2. 효율적 (불필요한 계산 생략)
- 3. 행동 전환 명확

단점:

- 1. 낮은 우선순위 행동 무시될 수 있음
- 2. 행동 전환이 급격할 수 있음

6.5.3 방법 3: 블렌딩 (Blending)

접근법: 두 행동 사이를 부드럽게 보간

구현:

```
Vector3 BlendBehaviors(Vector3 behavior1, Vector3 behavior2, float t)
{
    return Vector3.Lerp(behavior1, behavior2, t);
}

// 예시: 체력 기반 블렌딩
float healthRatio = currentHealth / maxHealth;

Vector3 attackForce = Pursue(enemy);
Vector3 fleeForce = Flee(enemy);

Vector3 finalForce = BlendBehaviors(fleeForce, attackForce, healthRatio);
// 체력 낮음: flee, 체력 높음: attack, 중간: 둘의 혼합

ApplyForce(finalForce);
```

6.6 연습문제

문제 6.1 [개념 이해]

- (a) Kinematic 이동과 Dynamic 이동의 차이를 설명하시오.
- (b) 각각 어떤 게임 장르에 적합한지 예를 들어 설명하시오.

문제 6.2 [수학적 계산]

에이전트의 위치가 (0, 0)이고 velocity 가 (3, 4)일 때:

- (a) 목표 (10, 0)을 향한 Seek 조향력을 계산하시오 ($\text{maxSpeed} = 5$)
- (b) 위치 (5, 5)로부터 Flee 조향력을 계산하시오

문제 6.3 [설계 문제]

- (a) Seek 와 Pursue 의 차이를 설명하시오.
- (b) 미래 위치 예측의 원리를 수식으로 나타내시오.

문제 6.4 [구현]

다음 행동을 Unity C#로 구현하시오:

- (a) Wander (무작위 배회): 원 위의 무작위 점을 향해 Seek
- (b) Interpose (사이 끼어들기): 두 타겟 사이 위치로 이동
- (c) Hide (숨기): 장애물 뒤로 은신

문제 6.5 [복합 조향]

호위 로봇 시나리오를 설계하시오:

- 평상시: 플레이어 5m 뒤 추종
- 적 발견: 플레이어와 적 사이 위치
- 장애물: 회피
- 동료 로봇: 충돌 방지

우선순위와 가중치를 명시하시오.

Part III 종합 연습문제

프로젝트 3.1: Boids 시뮬레이션

목표: 3D 물고기 떼 군집 구현

요구사항:

- 1. UI 슬라이더로 실시간 가중치 조정
- 2. 경계 제약 구현
- 3. 최소 50 개 개체 + 공간 분할 최적화
- 4. FOV 설정 가능

평가 기준:

- Boids 3 규칙 정확성: 30 점
- 공간 최적화: 20 점
- 파라미터 UI 제어: 20 점
- 시각적 품질: 20 점
- 코드 품질 및 문서화: 10 점

프로젝트 3.2: 추적 미사일 구현

목표: Pursue 알고리즘으로 플레이어 추적 유도 미사일

요구사항:

- 1. 플레이어 미래 위치 예측 추적
- 2. 비행 중 장애물 회피

- 3. 연료 제한 (5 초 수명)
- 4. 파티클 효과 및 사운드

보너스 도전:

- 예측 시간 조절로 회피 난이도 변경
- 다수 미사일 동시 발사 + Separation

Part IV. 경로 탐색 (Pathfinding)

게임 인공지능

NPC의 움직임과 판단, 학습을 만드는 13 가지 알고리즘 패턴

Part IV 개요

Part III에서는 Steering Behaviors를 통해 AI가 "어떻게 움직이는가"를 다뤘습니다.

하지만 "어디로 갈 것인가?"에 대한 질문이 남았습니다. 출발점과 목적지 사이에 벽, 건물, 강 등 수많은 장애물이 있을 때, AI는 어떻게 최적의 경로를 찾을까요?

경로 탐색(Pathfinding)은 게임 AI의 가장 오래되고 보편적인 문제입니다. 1980년대 팩맨의 유령부터 현대 엘든 링의 보스까지, 모든 게임 AI는 경로 탐색을 필요로 합니다.

Part IV에서는 그래프 이론의 기초부터 시작하여 A* 알고리즘의 수학적 원리를 이해하고, Unity C#로 구현하며, Navigation Mesh(NavMesh)를 활용한 실전 기법까지 학습합니다. 1968년 Stanford Research Institute에서 개발된 A*는 50년이 지난 지금도 게임 AI의 표준으로 사용되는 놀라운 알고리즘입니다.

학습 목표

본 Part 를 학습한 후 여러분은 다음을 수행할 수 있어야 합니다:

1. 그래프 이론의 기초(노드, 에지, 가중치)를 이해하고 게임 맵을 그래프로 모델링할 수 있다.
2. BFS, Dijkstra, A* 알고리즘의 진화 과정과 차이점을 비교 분석할 수 있다.
3. A* 알고리즘의 수학적 원리($f = g + h$)를 이해하고 Unity C#로 구현할 수 있다.
4. 휴리스틱 함수(맨해튼, 유클리디안, 옥타일)의 특성을 이해하고 상황에 맞게 선택할 수 있다.
5. Navigation Mesh 의 원리를 이해하고 Unity NavMesh 시스템을 활용할 수 있다.
6. 경로 스무딩, 동적 장애물, Off-Mesh Link 등 실전 기법을 적용할 수 있다.

Part IV 구조

Chapter	제목	주요 내용	분량
7	A* 알고리즘과 최적 경로 탐색	그래프 이론, BFS/Dijkstra, A* 구현, 휴리스틱, 최적화	~40p
8	Navigation Mesh	NavMesh 원리, Unity NavMesh, Off-Mesh Link, 동적 장애물	~30p

Chapter 7. A* 알고리즘과 최적 경로 탐색

7.1 그래프 이론 기초

7.1.1 그래프의 정의

정의 7.1 (그래프)

$$G = (V, E)$$

구성 요소:

- V: 노드(Vertex/Node)의 집합 $\{v_1, v_2, \dots, v_n\}$
- E: 에지(Edge)의 집합 $\{(v_i, v_j) \mid v_i, v_j \in V\}$

정의 7.2 (가중 그래프)

각 에지 $e \in E$ 에 비용(Cost/Weight) $w(e) \geq 0$ 부여

게임 맵을 그래프로 모델링

게임 요소	그래프 요소
이동 가능한 타일/위치	노드 (Vertex)
인접한 타일 간 연결	에지 (Edge)
이동 비용 (거리, 지형 난이도)	가중치 (Weight)
장애물/벽	에지 없음 (연결 불가)

그래프의 종류

종류	정의	게임 예시
무방향 그래프	에지에 방향이 없음 (양방향)	대칭적인 2D 맵
방향 그래프	에지에 방향이 있음	일방통행 도로
가중 그래프	에지에 비용이 있음	지형별 이동 속도 차이
비가중 그래프	모든 에지 비용이 동일	균일한 타일 맵

7.1.2 게임 맵의 그래프 표현 방법

방법 1: 인접 행렬 (Adjacency Matrix)

```
public class GraphMatrix
{
    private float[,] adjacencyMatrix;
    private int nodeCount;

    // Matrix[i][j] = {에지 가중치 (연결됨), ∞ (연결 안 됨)}
}
```

장점: 두 노드 간 연결 확인 $O(1)$, 구현이 간단

단점: 메모리 $O(N^2)$, 희소 그래프에서 낭비

방법 2: 인접 리스트 (Adjacency List)

```
public class GraphNode
{
    public Vector2Int position;
    public List<Edge> neighbors;
}

public class Edge
{
    public GraphNode target;
    public float cost;
}
```

장점: 메모리 $O(V + E)$, 희소 그래프에 적합

단점: 노드 간 연결 확인 $O(\text{degree})$, 이웃 수에 비례

게임에서의 선택:

- 격자 맵 (Grid): 인접 리스트 권장 (대부분 4~8 개 이웃만 존재)
- 완전 연결 그래프: 인접 행렬 고려

- NavMesh: 인접 리스트 필수 (폴리곤 수가 적고 이웃도 적음)

7.1.3 그래프 탐색 알고리즘의 진화

1. 너비 우선 탐색 (BFS - Breadth-First Search)

원리: 시작 노드에서 가까운 노드부터 차례대로 모든 노드를 방문

의사코드:

```
function BFS(start, goal):
    queue ← [start]
    visited ← {start}

    while queue is not empty:
        current ← queue.dequeue()

        if current == goal:
            return PATH_FOUND

        for each neighbor of current:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.enqueue(neighbor)

    return NO_PATH
```

특징:

- 모든 에지 비용이 동일할 때 최단 경로 보장
- 목적지와 반대 방향의 노드까지 모두 검사 → 비효율적
- 시간 복잡도: $O(V + E)$

시각적 설명:

시작(S)에서 BFS 탐색 순서:

$S \rightarrow 1 \text{ 층 이웃} \rightarrow 2 \text{ 층 이웃} \rightarrow 3 \text{ 층 이웃} \dots$

목표(G)가 오른쪽에 있어도 왼쪽, 위, 아래도 모두 탐색

2. 다익스트라 (Dijkstra) 알고리즘

원리: 간선에 비용(Cost/Weight) 개념을 도입. 시작점으로부터 누적 비용이 가장 적은 노드를 우선 탐색

정의 7.3 (다익스트라 알고리즘)

각 노드 v 에 대해 시작 노드 s 로부터의 최단 거리 $\text{dist}(v)$ 를 유지하며, dist 가 가장 작은 미방문 노드를 선택하여 확장

의사코드:

```
function DIJKSTRA(start, goal):
    dist[start] ← 0
    dist[all others] ← ∞
    priorityQueue ← [(0, start)]

    while priorityQueue is not empty:
        current ← priorityQueue.extractMin()

        if current == goal:
            return RECONSTRUCT_PATH

        for each neighbor of current:
            newCost ← dist[current] + cost(current, neighbor)

            if newCost < dist[neighbor]:
                dist[neighbor] ← newCost
                parent[neighbor] ← current
                priorityQueue.insert((newCost, neighbor))

    return NO_PATH
```

특징:

- 가중 그래프에서 최단 경로 보장

- 음수 가중치가 없을 때만 작동
- 여전히 목적지의 방향성을 고려하지 않고 사방으로 탐색 확장
- 시간 복잡도: $O((V + E) \log V)$ (우선순위 큐 사용 시)

BFS vs Dijkstra 비교

특성	BFS	Dijkstra
그래프 종류	비가중 그래프	가중 그래프
탐색 기준	깊이(레벨)	누적 비용
자료구조	큐 (Queue)	우선순위 큐
최단 경로 보장	모든 비용 = 1 일 때만	항상 (음수 없을 때)

3. A* 알고리즘: Dijkstra + 휴리스틱

핵심 아이디어:

"과거(실제 비용)과 미래(예상 비용)를 함께 고려하여 유망한 경로를 우선 탐색한다."

Dijkstra 는 최단 경로를 보장하지만 목적지 방향을 모르기 때문에 불필요한 방향으로도 탐색합니다. A*는 휴리스틱(Heuristic)을 결합하여 탐색 성능을 비약적으로 향상시킵니다.

7.2 A* 알고리즘 원리

7.2.1 핵심 공식: $f(n) = g(n) + h(n)$

정의 7.4 (A* 평가 함수)

$$f(n) = g(n) + h(n)$$

각 성분의 의미:

성분	정의	특성
g(n) (Actual Cost)	시작 노드에서 현재 노드 n 까지 이동하는 데 걸린 실제 비용	"과거의 정보" - 확정된 값
h(n) (Heuristic Cost)	현재 노드 n 에서 목표 노드까지 남은 거리의 추정 비용	"미래의 정보" - 추정값
f(n) (Total Cost)	시작점에서 목표점까지 가는 전체 예상 비용	$g(n) + h(n)$

시각적 이해:

시작(S) ----[g(n)]---- 현재(n) ----[h(n)]---- 목표(G)

(실제 이동 비용) (추정 남은 비용)

$f(n) = g(n) + h(n)$ = 전체 예상 경로 비용

예제 7.2: A* 비용 계산

시작: (0, 0)

현재: (2, 1)

목표: (5, 4)

$g(2,1)$ = 시작부터 (2,1)까지 실제 이동한 비용 = 3 (예: 3 칸 이동)

$h(2,1)$ = (2,1)에서 (5,4)까지 추정 거리

$$= \text{맨해튼 거리} = |5-2| + |4-1| = 3 + 3 = 6$$

$$f(2,1) = g(2,1) + h(2,1) = 3 + 6 = 9$$

A*의 유망성:

A*는 f 값이 가장 작은 노드를 우선 탐색하므로, "가장 유망해 보이는 경로"를 먼저 확인합니다.

7.2.2 A* 알고리즘 의사 코드

```
function A_STAR(start, goal):
    # 초기화
    openList ← PriorityQueue()
    closedList ← Set()

    g[start] ← 0
    h[start] ← HEURISTIC(start, goal)
    f[start] ← g[start] + h[start]
    openList.insert(start, f[start])

    # 메인 루프
    while openList is not empty:
        current ← openList.extractMin() # f 값이 가장 작은 노드

        # 목표 도달
        if current == goal:
            return RECONSTRUCT_PATH(current)

        closedList.add(current)

        # 이웃 노드 탐색
        for each neighbor of current:
            if neighbor in closedList:
                continue # 이미 탐색 완료

            tentative_g ← g[current] + COST(current, neighbor)

            if neighbor not in openList:
                openList.insert(neighbor, ∞)

            if tentative_g < g[neighbor]:
                # 더 나은 경로 발견
                parent[neighbor] ← current
                g[neighbor] ← tentative_g
                h[neighbor] ← HEURISTIC(neighbor, goal)
                f[neighbor] ← g[neighbor] + h[neighbor]
                openList.updatePriority(neighbor, f[neighbor])

    return NO_PATH # 경로 없음
```

알고리즘 동작 단계:

- 1. Open List (열린 목록): 탐색해야 할 후보 노드들을 f 값 기준으로 우선순위 큐에 저장
- 2. Closed List (닫힌 목록): 이미 탐색이 완료된 노드들
- 3. 루프: Open List 에서 f 값이 가장 낮은 노드를 꺼내서 처리
- 4. 목표 확인: 현재 노드가 목표면 경로 역추적
- 5. 이웃 탐색: 이웃 노드들의 g , h , f 계산 및 갱신

7.2.3 Unity C# 구현

노드 클래스:

```
public class AStarNode : IComparable<AStarNode>
{
    public Vector2Int position;
    public float g; // 시작부터 현재까지 실제 비용
    public float h; // 현재부터 목표까지 추정 비용
    public float f => g + h; // 전체 예상 비용
    public AStarNode parent; // 경로 역추적용

    public AStarNode(Vector2Int pos)
    {
        position = pos;
        g = float.MaxValue;
        h = 0;
        parent = null;
    }

    // 우선순위 큐를 위한 비교
    public int CompareTo(AStarNode other)
    {
        return f.CompareTo(other.f);
    }
}
```

A* 구현 (일부):

```
using System.Collections.Generic;
using UnityEngine;

public class AStarPathfinder
{
    private int width, height;
    private bool[,] walkable;

    public AStarPathfinder(int w, int h, bool[,] walkableMap)
    {
        width = w;
        height = h;
        walkable = walkableMap;
    }

    public List<Vector2Int> FindPath(Vector2Int start, Vector2Int goal)
    {
        Dictionary<Vector2Int, AStarNode> allNodes = new
            Dictionary<Vector2Int, AStarNode>();

        // Open List (우선순위 큐)
        SortedSet<AStarNode> openList = new SortedSet<AStarNode>(
            Comparer<AStarNode>.Create((a, b) => {
                int cmp = a.f.CompareTo(b.f);
                return cmp == 0 ?

a.position.GetHashCode().CompareTo(b.position.GetHashCode()) : cmp;
            })
        );

        HashSet<Vector2Int> closedList = new HashSet<Vector2Int>();

        // 시작 노드 초기화
        AStarNode startNode = new AStarNode(start);
        startNode.g = 0;
        startNode.h = Heuristic(start, goal);
        allNodes[start] = startNode;
        openList.Add(startNode);

        // 메인 루프
        while (openList.Count > 0)
        {
            AStarNode current = openList.Min;
            openList.Remove(current);
```

```

        if (current.position == goal)
        {
            return ReconstructPath(current);
        }

        closedList.Add(current.position);

        // 이웃 노드 탐색
        foreach (Vector2Int neighborPos in
GetNeighbors(current.position))
        {
            if (closedList.Contains(neighborPos))
                continue;

            if (!walkable[neighborPos.x, neighborPos.y])
                continue;

            float tentativeG = current.g + GetCost(current.position,
neighborPos);

            AStarNode neighbor;
            if (!allNodes.ContainsKey(neighborPos))
            {
                neighbor = new AStarNode(neighborPos);
                allNodes[neighborPos] = neighbor;
            }
            else
            {
                neighbor = allNodes[neighborPos];
            }

            if (tentativeG < neighbor.g)
            {
                if (openList.Contains(neighbor))
                    openList.Remove(neighbor);

                neighbor.parent = current;
                neighbor.g = tentativeG;
                neighbor.h = Heuristic(neighborPos, goal);
                openList.Add(neighbor);
            }
        }
    }

    return null; // 경로 없음
}

```

```

List<Vector2Int> ReconstructPath(AStarNode goal)

```

```

{
    List<Vector2Int> path = new List<Vector2Int>();
    AStarNode current = goal;
    while (current != null)
    {
        path.Add(current.position);
        current = current.parent;
    }
    path.Reverse();
    return path;
}

float GetCost(Vector2Int from, Vector2Int to)
{
    Vector2Int diff = to - from;
    if (Mathf.Abs(diff.x) + Mathf.Abs(diff.y) == 1)
        return 1f; // 상하좌우
    return 1.414f; // 대각선 (√2)
}

float Heuristic(Vector2Int from, Vector2Int to)
{
    return Mathf.Abs(from.x - to.x) + Mathf.Abs(from.y - to.y);
}
}

```

7.3 휴리스틱 함수 (Heuristics)

휴리스틱 함수 $h(n)$ 은 A*의 핵심입니다. 이를 어떻게 정의하느냐에 따라 A*의 성능과 정확도가 결정됩니다.

7.3.1 주요 휴리스틱 함수

1. 맨해튼 거리 (Manhattan Distance)

정의:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

특징:

- 격자(Grid) 맵에서 대각선 이동이 불가능할 때 사용
- 가로/세로 거리의 합
- 택시가 블록을 도는 것처럼 이동

적용 사례:

- 상하좌우만 이동 가능한 타일 맵
- RTS 게임의 유닛 이동 (대각선 금지)

Unity C# 구현:

```
float ManhattanDistance(Vector2Int from, Vector2Int to)
{
    return Mathf.Abs(from.x - to.x) + Mathf.Abs(from.y - to.y);
}
```

2. 유클리디안 거리 (Euclidean Distance)

정의:

$$h(n) = \sqrt{[(x_1 - x_2)^2 + (y_1 - y_2)^2]}$$

특징:

- 장애물이 없는 직선거리
- 대각선 이동이 자유로운 환경
- 3D 공간에 적합

Unity C# 구현:

```
float EuclideanDistance(Vector2Int from, Vector2Int to)
{
    return Vector2.Distance(from, to);
}

float EuclideanDistance3D(Vector3 from, Vector3 to)
{
    return Vector3.Distance(from, to);
}
```

3. 대각선 거리 (Diagonal Distance / Chebyshev)

정의:

$$h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$$

특징:

- 대각선 이동 비용 = 직선 이동 비용일 때 사용
- 8 방향 이동이 모두 비용 1

Unity C# 구현:

```
float DiagonalDistance(Vector2Int from, Vector2Int to)
{
    return Mathf.Max(Mathf.Abs(from.x - to.x),
                     Mathf.Abs(from.y - to.y));
}
```

4. 옥타일 거리 (Octile Distance)

정의:

대각선과 직선의 비용이 다를 때 (직선 = 1, 대각선 = $\sqrt{2}$):

$$h(n) = (dx + dy) + (\sqrt{2} - 2) \times \min(dx, dy)$$

여기서 $dx = |x_1 - x_2|$, $dy = |y_1 - y_2|$

Unity C# 구현:

```
float OctileDistance(Vector2Int from, Vector2Int to)
{
    int dx = Mathf.Abs(from.x - to.x);
    int dy = Mathf.Abs(from.y - to.y);
    return (dx + dy) + (1.414f - 2f) * Mathf.Min(dx, dy);
}
```

휴리스틱 비교표:

휴리스틱	이동 방식	비용 모델	게임 예시
맨해튼	상하좌우만	직선 = 1	격자 퍼즐, 일부 RTS
유클리디안	자유 이동	직선 거리	3D 게임, NavMesh
대각선	8 방향, 비용 동일	모두 = 1	레트로 RPG
옥타일	8 방향, 비용 다름	직선 = 1, 대각 = $\sqrt{2}$	현대 2D 게임

7.3.2 허용성 (Admissibility)과 일관성 (Consistency)

정의 7.5 (허용성 / Admissible Heuristic)

휴리스틱 $h(n)$ 이 실제 남은 비용을 절대 과대평가하지 않으면 허용 가능하다고 함:

$$h(n) \leq h^*(n)$$

여기서 $h^*(n)$ 은 n 에서 목표까지의 실제 최단 거리

중요성:

- $h(n)$ 이 허용 가능하면 A^* 는 항상 최단 경로를 찾습니다
- 과대평가하면 더 나은 경로를 놓칠 수 있습니다

정의 7.6 (일관성 / Consistent Heuristic)

모든 노드 n 과 그 이웃 n' 에 대해:

$$h(n) \leq c(n, n') + h(n')$$

여기서 $c(n, n')$ 은 n 에서 n' 로 가는 실제 비용

의미: "삼각 부등식"을 만족. 일관성은 허용성보다 강한 조건이며, 일관성을 만족하면 자동으로 허용성도 만족.

예제 7.3: 허용성 검증

실제 최단 거리 $h^*(n) = 10$

허용 가능:

- $h(n) = 8$ (과소평가)

- $h(n) = 10$ (정확)

허용 불가:

- $h(n) = 12$ (과대평가)

주요 휴리스틱의 허용성:

휴리스틱	허용성	이유
맨해튼 (4 방향 이동)	✓	직선 거리보다 항상 크거나 같음
유클리디안	✓	최단 직선 거리
대각선	조건부	대각 비용 = 1 일 때만
상수 0	✓	항상 과소평가 (Dijkstra 동일)

7.3.3 휴리스틱의 영향

$h(n) = 0$ 인 경우:

- A^* = Dijkstra 가 됨
- 모든 방향으로 균등하게 탐색
- 최단 경로를 보장하지만 느림

$h(n)$ 이 정확한 경우:

- 이상적으로 최단 경로만 탐색하여 매우 빠름
- 하지만 실제로는 불가능 (정확한 h 를 알면 경로를 이미 아는 것)

$h(n)$ 이 과소평가:

- 최단 경로 보장
- 하지만 불필요한 노드도 탐색하여 느림

$h(n)$ 이 과대평가:

- 빠르지만 최단 경로를 놓칠 수 있음

실험 결과 (100x100 맵, 장애물 30%, 맨해튼 거리):

$h(n) = 0$ (Dijkstra):

탐색 노드: 4,521 개

시간: 45ms

$h(n) = \text{맨해튼}$:

탐색 노드: 312 개

시간: 5ms

$h(n) = \text{맨해튼} \times 2$ (과대평가):

탐색 노드: 85 개

시간: 2ms

경로 길이: 152 (최단: 147) X

7.4 A* 최적화와 변형

7.4.1 Weighted A* (가중 A*)

정의 7.7 (Weighted A*)

휴리스틱에 가중치 $\varepsilon \geq 1$ 을 곱합니다:

$$f(n) = g(n) + \varepsilon \times h(n)$$

효과:

- $\varepsilon > 1$: 휴리스틱의 영향력 증가
 - 목표 방향 우선 탐색
 - 빠르지만 최적성 손실

$\varepsilon = 1$: 일반 A*

$\varepsilon \rightarrow \infty$: Greedy Best-First (h 만 고려)

Unity C# 구현:

```
public class WeightedAStar : AStarPathfinder
{
    public float epsilon = 1.5f; // 가중치

    protected override float CalculateF(float g, float h)
    {
        return g + epsilon * h;
    }
}
```

정리 7.1 (ε -admissibility)

Weighted A*는 ϵ -최적을 보장합니다:

$$cost(found_path) \leq \epsilon \times cost(optimal_path)$$

즉, 최대 ϵ 배 길 수 있지만 그 이상은 아님

게임 적용:

- RTS 게임: 빠른 응답성이 중요할 때 $\epsilon = 1.2 \sim 1.5$ 사용
- 턴제 게임: 정확성이 중요하므로 $\epsilon = 1.0$ 유지

7.4.2 Jump Point Search (JPS)

개념: 격자 맵에서 불필요한 중간 노드들을 건너뛰고, 코너나 교차로 같은 핵심
지점(Jump Point)만 탐색하여 성능을 극대화

핵심 아이디어:

직선 경로상의 모든 노드를 검사하지 않고, 방향이 바뀌는 지점(Jump Point)만 탐색

Jump Point 정의: 다음 중 하나를 만족하는 노드

- 1. 목표 노드
- 2. 강제 이웃(Forced Neighbor)이 있는 노드
- 3. 대각선 이동 시, 수평/수직 방향에 Jump Point 가 있는 노드

성능 (100x100 맵 비교):

알고리즘	탐색 노드 수	상대 속도
A* (맨해튼)	1,000	1.0x
A* (옥타일)	850	1.2x
JPS	120	8.3x

제한사항:

- 균일한 격자 맵에서만 작동
- 지형 비용이 다르면 사용 불가
- NavMesh 에는 적용 불가

적용 사례:

- 2D 타일 기반 게임 (스타크래프트 리마스터)
- 로그라이크 던전
- 타워 디펜스

7.4.3 계층적 경로 탐색 (Hierarchical Pathfinding)

개념: 거대한 맵을 작은 지역(Region)으로 나누고, 먼저 고수준(지역 간)에서 경로를 찾은 후 저수준(지역 내)에서 세부 경로를 찾음

알고리즘:

- 1. 맵을 여러 클러스터(Cluster)로 분할
- 2. 각 클러스터 간 연결점(Entry Point) 정의
- 3. 고수준 그래프에서 A* 실행 (클러스터 간 경로)
- 4. 각 클러스터 내부에서 A* 실행 (세부 경로)

장점:

- 대규모 맵에서 메모리 절약
- 탐색 속도 향상 (계층 분할로 $O(N) \rightarrow O(\sqrt{N})$)

적용 사례:

- 오픈 월드 게임
- 멀티플레이어 MOBA (리그 오브 레전드)
- MMO RPG

7.4.4 동적 A* (Dynamic A* / D*)

개념: 맵이 동적으로 변할 때 전체를 다시 계산하지 않고, 변화된 부분만 업데이트

D* Lite 알고리즘: 로봇 공학에서 개발된 알고리즘으로, 에이전트가 이동하면서 새로운 정보를 얻을 때 효율적으로 경로를 재계산

게임 적용:

- 문이 열리거나 닫힐 때
- 다리가 무너질 때
- 다른 유닛이 길을 막을 때

Chapter 8. Navigation Mesh (내비게이션 메시)

8.1 Navigation Mesh 원리

8.1.1 Grid vs NavMesh

Grid 기반 접근의 한계:

3D 게임 맵 (50m × 50m × 10m 높이)

Grid 해상도: 0.5m

필요한 셀 수: $100 \times 100 \times 20 = 200,000$ 개

메모리: $200,000 \times 4 \text{ bytes} = 800 \text{ KB}$ (비용만)

문제점:

1. 넓은 평지도 수천 개의 셀로 분할
2. 경사, 계단 등 3D 지형 표현 어려움
3. 다층 구조 처리 복잡
4. A* 탐색 시 노드 수가 너무 많음

NavMesh 접근:

동일한 맵을 폴리곤으로 표현:

폴리곤 수: 약 500 개

메모리: 훨씬 적음 (정점 좌표만 저장)

장점:

1. 넓은 평지 = 하나의 큰 폴리곤
2. 지형 형태 그대로 표현

3. 다층 자동 처리

4. A* 탐색 노드 수 급감 (200,000 → 500)

8.1.2 폴리곤 기반 표현

정의 8.1 (Navigation Mesh)

Navigation Mesh 는 이동 가능한 영역(Walkable Area)을 볼록 다각형(Convex Polygon)들의 집합으로 표현한 그래프입니다:

- 노드: 각 폴리곤
- 에지: 폴리곤들이 공유하는 변(Edge)
- 가중치: 폴리곤 간 이동 비용

볼록 다각형 (Convex Polygon):

다각형 내부의 임의의 두 점을 잇는 선분이 항상 다각형 내부에 있는 다각형

왜 볼록 다각형인가?

- 1. 직선 이동 보장: 폴리곤 내부에서는 어느 지점이든 직선으로 이동 가능
- 2. 간단한 충돌 검사: 점이 폴리곤 내부에 있는지 빠르게 판단
- 3. 경로 계산 용이: 폴리곤 중심이나 모서리를 연결하여 경로 생성

NavMesh 생성 과정:

1. 지형 분석

- 이동 가능 표면 감지
- 경사도 체크 (예: 45 도 이상은 이동 불가)
- 에이전트 크기 고려 (좁은 틈은 제외)

2. 복셀화 (Voxelization)

- 3D 공간을 작은 복셀(Voxel)로 분할
- 이동 가능 복셀 마킹

3. 폴리곤 생성

- 복셀을 병합하여 큰 폴리곤 생성
- 오목한 부분을 여러 볼록 폴리곤으로 분할

4. 최적화

- 불필요한 정점 제거
- 이웃 폴리곤 병합

8.1.3 Unity NavMesh 시스템

NavMesh 구성 요소:

- 1. NavMesh Surface: 이동 가능 영역 정의
- 2. NavMesh Agent: 경로를 따라 이동하는 AI
- 3. NavMesh Obstacle: 동적 장애물
- 4. Off-Mesh Link: 특수 연결 (점프, 사다리 등)

기본 설정 (Unity C#):

```
using UnityEngine;
using UnityEngine.AI;

public class NavMeshAgentController : MonoBehaviour
{
    public Transform target;
    private NavMeshAgent agent;

    void Start()
    {
        agent = GetComponent<NavMeshAgent>();

        // 에이전트 설정
        agent.speed = 5f; // 이동 속도
        agent.angularSpeed = 120f; // 회전 속도
        agent.acceleration = 8f; // 가속도
        agent.stoppingDistance = 1f; // 목표 근처에서 정지 거리
        agent.autoBraking = true; // 자동 감속
    }

    void Update()
    {
        if (target != null)
        {
            agent.SetDestination(target.position);
        }

        // 목표 도달 확인
    }
}
```

```
        if (!agent.pathPending && agent.remainingDistance < 0.5f)
        {
            Debug.Log("목표 도달!");
        }
    }
}
```

NavMesh Baking (굽기):

Unity 에디터에서:

1. Navigation 윈도우 열기 (Window → AI → Navigation)
2. Bake 탭 선택
3. 파라미터 설정:
 - Agent Radius: 에이전트 반지름 (0.5)
 - Agent Height: 에이전트 높이 (2.0)
 - Max Slope: 최대 경사각 (45 도)
 - Step Height: 오를 수 있는 계단 높이 (0.4)
4. Bake 버튼 클릭

8.2 NavMesh 고급 기능

8.2.1 경로 스무딩 (Path Smoothing)

문제: NavMesh 위에서 A*로 찾은 경로는 폴리곤의 중심이나 모서리를 연결한 것이라 지그재그 형태를 띌 수 있음

Funnel Algorithm (깔때기 알고리즘)

폴리곤 경로를 직선화하여 불필요한 꺾임을 제거

핵심 아이디어:

폴리곤 경로: $P1 \rightarrow P2 \rightarrow P3 \rightarrow P4$

각 폴리곤 간 공유 에지를 "포털(Portal)"로 간주

출발점에서 시작하여 포털들을 통과하는 최단 직선 경로 계산

알고리즘:

1. 시작점부터 첫 포털까지 깔때기 초기화
2. 다음 포털을 순회하며:
 - 포털이 깔때기 내부에 있으면 깔때기 좁히기
 - 포털이 깔때기 밖이면 꺾인 점(Apex) 저장하고 깔때기 재설정
3. 꺾인 점들을 연결한 경로가 최종 경로

Unity에서는 자동 처리:

NavMeshAgent 는 내부적으로 Funnel Algorithm 을 사용하여 자동으로 경로를 스무딩

수동 경로 접근:

```
NavMeshPath path = new NavMeshPath();
agent.CalculatePath(target.position, path);

if (path.status == NavMeshPathStatus.PathComplete)
{
    Vector3[] corners = path.corners;
    Debug.Log($"경로 코너 수: {corners.Length}");

    // 경로 시각화
    for (int i = 0; i < corners.Length - 1; i++)
    {
        Debug.DrawLine(corners[i], corners[i + 1], Color.green, 2f);
    }
}
```

8.2.2 Off-Mesh Link (오프 메시 링크)

개념: 떨어져 있는 두 NavMesh 영역 사이를 연결하는 특수한 링크

사용 사례:

- 점프 가능한 틈
- 사다리
- 텔레포트
- 로프 타기
- 창문을 통한 진입

Unity 구현:

```
public class OffMeshLinkHandler : MonoBehaviour
{
    private NavMeshAgent agent;

    void Start()
    {
        agent = GetComponent<NavMeshAgent>();
    }

    void Update()
    {
        // Off-Mesh Link 를 횡단 중인지 확인
        if (agent.isOnOffMeshLink)
        {
            StartCoroutine(HandleOffMeshLink());
        }
    }

    IEnumerator HandleOffMeshLink()
    {
        // 에이전트 자동 이동 중지
        agent.isStopped = true;
        OffMeshLinkData linkData = agent.currentOffMeshLinkData;
```

```

Vector3 startPos = linkData.startPos;
Vector3 endPos = linkData.endPos;
float duration = 0.5f; // 점프 시간
float elapsed = 0f;

// 수동으로 점프 애니메이션
while (elapsed < duration)
{
    float t = elapsed / duration;

    // 포물선 점프
    Vector3 position = Vector3.Lerp(startPos, endPos, t);
    position.y += Mathf.Sin(t * Mathf.PI) * 2f; // 높이 2m

    transform.position = position;
    elapsed += Time.deltaTime;
    yield return null;
}

transform.position = endPos;

// Off-Mesh Link 완료
agent.CompleteOffMeshLink();
agent.isStopped = false;
}
}

```

8.2.3 NavMesh Area 와 비용

개념: 서로 다른 지형 타입에 다른 이동 비용을 부여할 수 있음

Area 정의:

Navigation → Areas:

- Walkable (비용: 1)
- Road (비용: 0.5) - 빠른 이동
- Grass (비용: 2) - 느린 이동
- Water (비용: 5) - 매우 느림
- Lava (비용: ∞) - 이동 불가

에이전트별 Area Mask:

```
NavMeshAgent agent = GetComponent<NavMeshAgent>();

// 특정 Area 만 사용
int walkableMask = 1 << NavMesh.GetAreaFromName("Walkable");
int roadMask = 1 << NavMesh.GetAreaFromName("Road");
agent.areaMask = walkableMask | roadMask;

// 물은 건너지 않음 (Water 제외)
```

Area 비용 조절:

```
// 전역 Area 비용 설정
NavMesh.SetAreaCost(NavMesh.GetAreaFromName("Water"), 10f);

// 에이전트별 비용 설정
agent.SetAreaCost(NavMesh.GetAreaFromName("Grass"), 3f);
```

전략적 활용:

- 적 AI: 도로 우선 (빠른 추격)
- 잠입 NPC: 숲 우선 (은신)
- 탱크 유닛: 평지만 이동
- 비행 유닛: 모든 지형 무시

8.3 동적 NavMesh (Dynamic NavMesh)

8.3.1 문제 상황

게임 중 발생 가능한 변화:

- 문이 열리거나 닫힘
- 건물이 무너짐
- 다리가 건설됨
- 폭발로 지형 파괴

문제: 전체 NavMesh 를 다시 Bake 하는 것은 비용이 크므로 (수 초 소요), 실시간

게임에서는 불가능

8.3.2 NavMesh Obstacle (Carving)

개념: 동적 장애물 추가

Unity 구현:

```
public class DynamicDoor : MonoBehaviour
{
    public bool isOpen = false;
    private NavMeshObstacle obstacle;

    void Start()
    {
        obstacle = GetComponent<NavMeshObstacle>();
        obstacle.carving = true; // NavMesh 에 구멍 뚫기 활성화
        UpdateObstacle();
    }

    public void ToggleDoor()
    {
        isOpen = !isOpen;
        UpdateObstacle();
    }

    void UpdateObstacle()
    {
        // 문이 열리면 장애물 제거
        obstacle.enabled = !isOpen;
    }
}
```

Carving 원리:

- 1. NavMesh Obstacle 에 carving = true 설정
- 2. 장애물이 NavMesh 에 구멍을 뚫음 (실시간)
- 3. 에이전트는 자동으로 새 경로 계산

주의사항:

- Carving 은 성능 비용 있음 (장애물이 많으면 프레임 드롭)
- 최대 장애물 수 제한 (보통 100~200 개)
- 정적 환경은 Bake, 동적만 Obstacle 사용

8.4 Grid A* vs NavMesh 비교

종합 비교표:

비교 항목	Grid 기반 A*	NavMesh
적합 환경	2D 게임, RTS, 단순 지형	3D FPS/RPG, 복잡한 다층 구조
메모리 효율	맵이 클수록 급증	넓은 영역을 적은 폴리곤으로 표현
정밀도	격자 단위로 제한적	기하학적으로 정밀한 이동 가능
구현 난이도	비교적 쉬움	Baking 알고리즘 복잡함
동적 변화	셀 단위로 쉽게 수정	Obstacle Carving 필요
탐색 속도	노드 수 많으면 느림	폴리곤 수 적어 빠름
3D 지형	층별로 별도 처리	자동으로 3D 처리
경로 품질	계단식, 각진 경로	부드러운 곡선 경로

선택 기준:

Grid A*를 선택:

- 2D 타일 기반 게임
- 턴제 전략 게임
- 지형이 단순하고 격자 구조가 자연스러운 경우
- 직접 경로 탐색 로직 제어 필요

NavMesh 를 선택:

- 3D 액션/어드벤처 게임
- 복잡한 지형과 다층 구조
- 대규모 오픈 월드
- Unity/Unreal 등 엔진 기능 활용 가능

